



Confidential VM Extension I/O (CoVE-IO) for Confidential Computing on RISC-V platforms

Editor - Samuel Ortiz, Jiewen Yao, RISC-V AP-TEE-IO Task Group

Version v0.3.0, 2026-02-18: This document is in development. Assume everything can change. See <http://riscv.org/spec-state> for details.

Table of Contents

Preamble	1
Copyright and license information	2
Contributors	3
1. Introduction	4
2. Glossary	6
3. Requirements	9
3.1. Device	9
3.1.1. PCIe or CXL.io	9
TEE Device Interface Protocol (TDISP)	9
Data Object Exchange (DOE)	9
Integrity and Data Exchange (IDE)	10
3.2. Host	11
3.2.1. Ownership	11
3.2.2. Platform Hardware Components	11
PCIe	11
IO Translation Agent	12
CoVE-IO Manifest	13
3.2.3. Software	13
Host	13
TSM	13
RDSM	14
3.3. Guest	14
4. Security Model	15
5. Threat Model	16
5.1. Assets	16
5.1.1. Security Objectives	16
5.2. Adversary Model	16
5.3. Threats	17
5.3.1. CoVE-IO-T001 - Trusted MMIO Malicious Access	17
5.3.2. CoVE-IO-T002 - Trusted MMIO Remapping	17
5.3.3. CoVE-IO-T003 - Trusted MMIO PCIe Redirection	18
5.3.4. CoVE-IO-T004 - Trusted MMIO PCIe Pre-Configuration	18
5.3.5. CoVE-IO-T005 - Trusted MMIO Unauthorized Access	19
5.3.6. CoVE-IO-T006 - PCIe Link Man-In-The-Middle	19
5.3.7. CoVE-IO-T007 - PCIe ID Spoofing	19
5.3.8. CoVE-IO-T008 - Confused Deputy DMA Remapping	20
5.3.9. CoVE-IO-T009 - DMA Remapping	20
5.3.10. CoVE-IO-T010 - DMA Remapping	21
5.4. Requirements	21
6. Architectural Overview	22
7. Theory of Operations	24
7.1. Platform Management	24
7.1.1. IOMMU Assignment	24

7.1.2. PCIe Root Port Management	24
PCIe Root Port IDE Key Programming.....	25
PCIe Root Port IDE Extended Capability	25
7.2. Device Initialization.....	25
7.3. SPDMM Transport.....	25
7.3.1. Secure SPDMM Session.....	28
7.4. Device Connection.....	28
7.4.1. SPDMM Session.....	29
7.4.2. Device Connection Transcript	31
TVM Verification Process	31
7.4.3. IDE Link	32
7.5. Device Disconnection.....	34
7.5.1. Disconnection Flow.....	35
7.6. Interface Binding.....	36
7.6.1. Binding Flow	37
7.6.2. TDI Verification and Acceptation.....	38
7.7. Interface Unbinding	42
7.7.1. Unbinding Flow.....	43
7.8. Session Key Update.....	44
7.8.1. SPDMM Session Key Update.....	44
7.8.2. IDE Session Key Refresh.....	45
7.9. Session Heartbeat.....	45
7.10. Device and Interface Lifecycle.....	45
8. Device Attestation.....	48
8.1. TVM attesting the Device	48
8.1.1. Local Verification	49
8.1.2. Remote Verification.....	49
8.1.3. Device Runtime Update and Reattestation	49
SPDMM Session TerminationPolicy	50
TDISP NO_FW_UPDATE	50
Post-update reattestation.....	50
Pre-update attestation.....	51
8.2. Third party attesting the TVM with the device.....	51
8.2.1. Local Verification	51
8.2.2. Remote Verification.....	51
9. Confidential VM Extension (CoVE) IO ABI.....	53
9.1. CoVE IO Function IDs Namespace	53
9.2. CoVE IO Host Extension (EID #0x434F5648 "COVH").....	53
9.2.1. Physical Device Management.....	53
Function: CoVE Host Connect Device (FID #1024).....	53
Function: CoVE Host Disconnect Device (FID #1025).....	54
9.2.2. TVM Memory Management.....	55
Function: CoVE Host Add TVM Interface Region (FID #1026).....	55
Function: CoVE Host Reclaim TVM Interface Region (FID #1027).....	55
9.2.3. Device Interface Management	56

Function: CoVE Host Bind Interface (FID #1028).....	56
Function: CoVE Host Unbind Interface (FID #1030).....	56
Function: CoVE Host SPDM Key Update (FID #1031)	57
Function: CoVE Host IDE Key Refresh (FID #1032)	57
Function: CoVE Host SPDM Heartbeat (FID #1033)	57
Function: CoVE Host Device Connection Transcript (FID #1034)	58
9.3. CoVE IO Guest Extension (EID #0x434F5647 "COVG")	58
9.3.1. Physical Device Query.....	58
Function: CoVE Guest Get Device Link (FID #1024)	58
Function: CoVE Guest Get Device Connection Transcript (FID #1025).....	59
Function: CoVE Guest Get Device Measurements (FID #1026)	60
9.3.2. Device Interface Management.....	61
Function: CoVE Guest Get Interface Report (FID #1027)	61
Function: CoVE Guest Get Interface State (FID #1028)	62
Function: CoVE Guest Map Interface MMIO (FID #1029)	62
Function: CoVE Guest Start Interface (FID #1030)	63
Function: CoVE Guest Stop Interface (FID #1031).....	63
9.4. CoVE IO TSM Extension (EID #0x434F5654 "COVT")	64
9.4.1. PCIe Root Port IDE Management.....	64
Function: CoVE TSM Program PCIe Root Port IDE Key (FID #1024).....	64
Function: CoVE TSM Clear PCIe Root Port IDE Key (FID #1025)	65
Function: CoVE TSM Enable PCIe Root Port IDE Key (FID #1026).....	66
Function: CoVE TSM Disable PCIe Root Port IDE Tx Key (FID #1027)	66
Function: CoVE TSM PCIe Root Port IDE Key Status (FID #1028).....	67
Function: CoVE TSM Program PCIe Root Port IDE Stream Extended Capability (FID #1029).....	68
10. Summary Listing of CoVE-IO Functions.....	69
10.1. Summary of CoVE IO Host Extension (COVH).....	69
10.2. Summary of CoVE IO Guest Extension (COVG).....	69
10.3. Summary of CoVE IO TSM Extension (COVT)	70
References.....	71

Preamble



This document is in the *Development state*

Assume everything can change. This draft specification will change before being accepted as standard, so implementations made to this draft specification will likely not conform to the future standard.

Copyright and license information

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). The full license text is available at creativecommons.org/licenses/by/4.0/.

Copyright 2023 by RISC-V International.

Contributors

This RISC-V specification has been contributed to directly or indirectly by (in alphabetical order):

Jiewen Yao (Editor), Kevin Broch, Osman Koyuncu, Ravi Sahita, Samuel Ortiz (Editor), Steven Bellock, Vedvyas Shanbhogue

Chapter 1. Introduction

The RISC-V Confidential VM Extension [CoVE] specification provides application and virtualized workloads with data confidentiality and integrity, addressing one of the major datacenter security challenges. By building a minimized Trusted Computing Base (TCB), the CoVE interfaces manage to isolate workloads from each others but also from untrusted domain host software components (e.g. the hypervisor). CoVE implementations create a Confidential Computing ([CC]) framework that allows for mitigating the risks that guest owners are exposed to when running their workloads on shared infrastructures like e.g. public clouds.

The value of such hardware-based Trusted Execution Environments (TEEs) is acknowledged across the industry, and Confidential Computing providers are looking for ways to improve the technology's scalability and performance. With the growing need for securing data processing, expanding Confidential Computing guests TCBs with trusted devices is of particular interest. Data analytics and transformation, artificial intelligence, financial transactions processing are only a few examples of confidential workloads for which a secure and performant I/O architecture is key to their operations. By extending guests' TCBs with trusted accelerators, NICs, or GPUs, such workloads would fully take advantage of their infrastructure provider's capacities while keeping their data protected.

The CoVE interfaces provide TEE Virtual Machines (TVMs) memory with confidential attributes and allow TVM guests to share parts of their address space with an untrusted domain. Although this enables confidential guests to be assigned with devices, directly or not, through para-virtualized I/O implementation, it comes with a significant performance cost. Without additional protection, TVMs have no ways to trust hypervisor-assigned devices and must exclude them from their TCB by not allowing them to directly access confidential memory. Consequently, with the current CoVE specification, data flowing from a device to a TVM must first go through dedicated shared memory regions for the confidential guest to then move it over to its confidential address space. This systematic data copy between shared and confidential memory is called bounce-buffering and can have a major performance impact for confidential workloads.

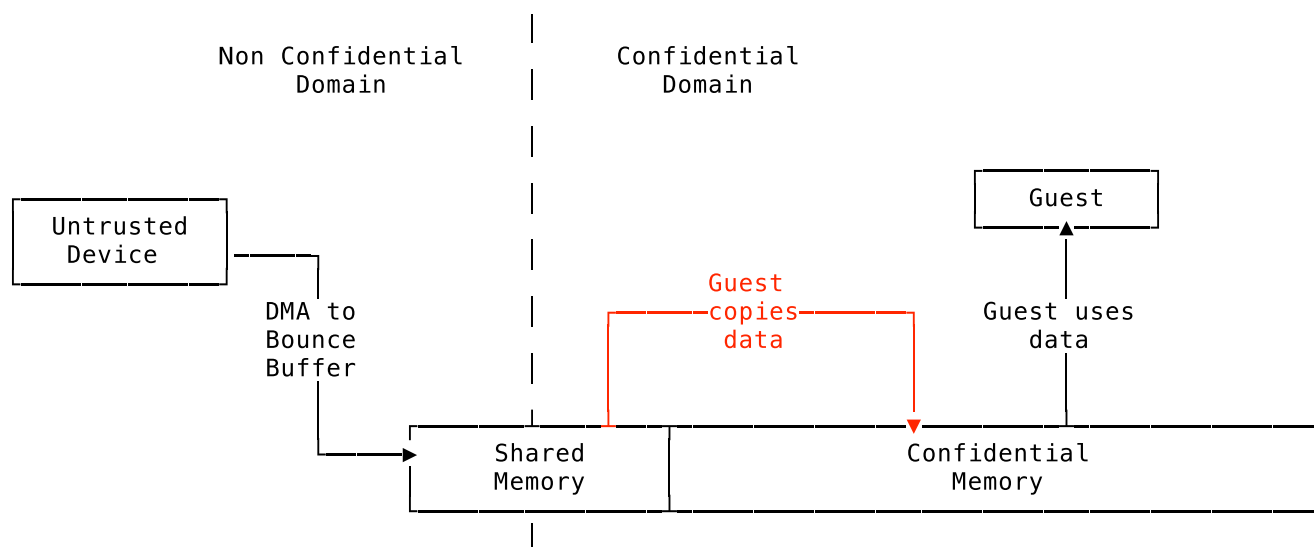


Figure 1. Bounce buffering between an untrusted device and a CoVE TVM

As devices typically expose their programming interfaces through memory mapped registers, using a shared memory buffer requires additional protection of the communication between the device and the TVM (e.g. transport-level data encryption). Such additional security layers can be impractical, intrusive, and may also degrade I/O performance even further.

Ideally, a secure and performant CoVE I/O framework would rely on the ability for hypervisor-assigned devices to directly access TVMs confidential memory, while maintaining the guest confidentiality and

integrity protection already provided by the CoVE security model. Building such a framework requires enhancing both the host software stack and the assigned devices with new protection mechanisms. In addition to the existing CoVE defined capabilities, the host software must provide ways for TVM guests to establish trust with assigned devices before accepting them into their TCBs and giving them direct access to confidential memory regions. Devices, on the other side of the I/O link, must protect confidential guest workloads and their data from untrusted domain components controlling, accessing, or tampering with them.

This document describes a proposal for extending the CoVE specification with I/O specific flows, interfaces, and intrinsics with the goal of implementing the above-described framework. The CoVE I/O interfaces aim at giving CoVE TVM guests the ability to securely:

- Retrieve a device's identity, configuration, and security state in order for them to establish trust with the device.
- Verify that any untrusted domain component will not be able to intercept, modify, or control data flowing between a guest and its assigned devices.
- Decide to accept devices into their TCB before allowing them to directly access their confidential memory, and before being able to control and configure said devices.

The CoVE I/O framework builds on top of the industry supported and ratified [\[PCI-SIG\]](#) TEE Device Interface Security Protocol ([\[TDISP\]](#)) specification, which itself relies on the [\[DMTF\]](#) Security Protocol and Data Model ([\[SPDM\]](#)) protocol. TDISP compliant devices, also known as TEE-IO devices, implement security protections for isolating guest workloads and confidential data from domains to which a device interface is not assigned to. It also requires TEE-IO devices to provide secure means for confidential guests to attest of any device interface trustworthiness and verify its security configuration.

Chapter 2. Glossary

Term	Acronym	Definition
Application Processor	AP	APs can support commodity operating systems, hypervisors/VMMs, and applications software workloads. The AP subsystem may contain several processing units, on-chip caches, and other controllers for interfacing with memory, accelerators, and other fixed-function logic. Multiple APs may be used within a logical system.
Application Processor- Trusted Execution Environment	AP-TEE	An execution mode that provides HW-isolation for workload assets when in use (user/supervisor code/ data) and provides HW-attestable confidentiality and integrity protection against specific attack vectors per a specified adversary and threat model. The term CoVE, TEE, and hardware-based TEE are also used as synonyms of AP-TEE in this document.
Address Translation Cache	ATC	A virtual to physical address translation cache in a PCIe device.
Address Translation Service	ATS	A PCIe protocol to support Device ATC.
Attestation	N/A	The process by which a relying party can assess the security posture of the confidential workload based on verifying a set of HW-rooted cryptographically-protected evidence.
Confidential Computing	N/A	The protection of data in use by performing computation in a hardware-based TEE.
Confidential VM	CVM	Same as a TVM.
Confidential VM Extension	CoVE	A set of non-ISA RISC-V extensions that enables confidential computing on RISC-V platforms.
CoVE Guest SBI Extension	COVG	CoVE SBI extensions serviced by the TSM and called by TVMs.
CoVE Host SBI Extension	COVH	CoVE SBI extensions serviced by the TSM and called by host software components.
Device Interface	DI	TDISP term, Device Interface. Same as a TDI. A DI can be a Virtual Function (VF) or a Physical Function (PF).
Device Security Manager	DSM	A DSM is a logical entity on a TEE-IO device that enforces the TDISP security policies, attributes, and states.
Direct Memory Access	DMA	The ability for a device to directly read and write into the host physical memory.
Host platform	N/A	The combination of the host software stack and the host hardware components. The host hardware components include all SoC components (CPU, memory controller, power management IP blocks, etc) and all discrete ones (Root of Trust, IOMMU, physical memory, etc).
Host software	N/A	All software elements including type-1 or type-2 HS-mode VMM and OS; U mode user-space VMM tools; ordinary VMs hosted by the VMM that emulate devices. The hosting platform is typically a multi-tenant platform that hosts multiple mutually distrusting Tenants.
Hypervisor or Virtual Machine Monitor	VMM	HS mode software that manages Virtual Machines by virtualizing hart, guest physical memory and IO resources. This document uses the term VMM and hypervisor interchangeably for this software entity.

Term	Acronym	Definition
Input/Output Memory Management Unit	IOMMU	A system agent that translates device virtual addresses to physical addresses.
Integrity and Data Encryption	IDE	Extended PCIe capability for integrity, confidentiality, and replay protection of PCIe Transport Layer Packets (TLP).
I/O Translation Agent	N/A	Same as an IOMMU.
Platform TCB	TCB	Platform Trusted Computing Base, same as TCB.
Physical Device	N/A	The physical device to which DIs belong to. This is the actual physical PCIe device.
Root Domain Security Manager	RDSM	The Root Domain Security Manager is an M-mode component that has access to all of the physical address space and resources on a platform. The RDSM enforces the isolation between security domains by controlling the access to the platform MTT and IO-MTT interfaces.
Root of Trust	ROT	Isolated HW/SW subsystem with an immutable ROM firmware and isolated compute and memory elements that form the Trusted Compute Base of a TEE system. The RoT manages cryptographic keys and other security critical functions such as system lifecycle and debug authorization. The RoT provides trusted services to other software on the platform such as verified boot, key provisioning and management, security lifecycle management, sealed storage, device management, cryptography services, attestation, etc. The RoT may be an integrated or discrete element, and may take on the role of a Device Identifier Composition Engine (DICE).
Supervisor Domain	N/A	A Supervisor Domain is associated with a set of physical address regions that are isolated from other Supervisor Domains on the same platform, with only the RDSM with access to all of the physical address space.
Supervisor Domain Security Manager	SDSM	The Supervisor Domain Security Manager is responsible for isolating workloads within a Supervisor Domain. SDSMs are S-Mode components like the TSM or the OS/Hypervisor.
Security Protocol and Data Model	SPDM	A DMTF defined specification for exchanging messages between devices over a variety of transports and physical media. In the CoVE-IO context, SPDM is used to exchange TDISP and IDE Key Management messages over PCIe DOE mailboxes.
Single-Root I/O Virtualization	SR-IOV	A PCIe specification that allows a single physical PCIe device (such as a network interface card, GPU, or storage controller) to present itself as multiple separate virtual devices to the system. This enables direct hardware access from virtual machines without requiring hypervisor intervention for I/O operations.
System TCB	TCB	System Trusted Computing Base, same as TCB.
TEE Device Interface Security Protocol	TDISP	A PCIe specified architecture for managing TEEs that include a composition of resources from one or more devices.
TEE I/O Device Interface	TDI	The unit of assignment for a trusted I/O capable device. For example, a TDI can be a Virtual Function (VF) or a Physical Function (PF).
TEE Security Manager	TSM	HS-mode software module that acts as the trusted (in TCB) intermediary between the VMM and the TVM. This module extends the TCB chain on the CoVE platform.
Transaction Layer Packet	TLP	The fundamental unit of data transfer between PCIe devices, root ports and switches.

Term	Acronym	Definition
Trusted Computing Base	TCB	The hardware, software, and firmware elements that are trusted by a relying party to protect the confidentiality and integrity of the relying parties' workload data and execution against a defined adversary model. In a system with separate processing elements within a package on a socket, the TCB boundary is the package. In a multi-socket system the TCB extends across the socket-to-socket interface, and is managed as one system TCB.
Trusted Device Manager	TDM	A CoVE-IO device manager, responsible for verifying, attesting, and accepting CoVE-IO devices into a TVM TCB. This is a TVM guest software stack component.
Trusted Execution Environment	TEE	An environment, which may include a composition of zero or more devices, within which some level of “trust” is established, such that operations (including code execution) is considered trustworthy. TEEs are isolated from untrusted environments by a TSM. A CoVE virtual machine is one example of a TEE.
Trusted Memory Mapped Input Output	Trusted MMIO	A TDI memory mapped I/O region that can only be accessed by a TVM that accepted the TDI in its TCB. TDIs describe their trusted MMIO regions through TDISP. The TVM, with the TSM support, is responsible for verifying that trusted MMIO ranges are correctly mapped into its address space.
TEE VM	TVM	A VM instantiation of a TEE.
Virtual Machine	VM	Virtual Machines hosted by a VMM

Chapter 3. Requirements

In order to extend TVM TCBs with external and untrusted devices, both the host platform and the assigned devices must meet a specific set of requirements.

3.1. Device

CoVE-IO compliant implementations support extending TVMs TCBs with devices if and only if they meet the following requirements:

3.1.1. PCIe or CXL.io

A CoVE-IO supported device must be a PCIe or CXL.io compliant device with optional Single-Root I/O Virtualization (SR-IOV) capabilities. When supporting SR-IOV, the device can be split into multiple Virtual Functions (VFs). A TVM-assigned interface can be the whole device, also known as the Physical Device (PF), or any of its VFs. In both cases, this unit of assignment is referred to as a TEE I/O Device Interface (TDI).

A host enumerated TDI must provide the following PCIe capabilities:

TEE Device Interface Protocol (TDISP)

In order to interoperate with a RISC-V CoVE-IO implementation, PCIe devices must support the [\[TDISP\]](#) protocol version 1.0 or above, as defined in the latest [\[PCIe\]](#) specification. CoVE-IO devices must support all required TDISP requests and may support the optional ones, as defined by the TDISP request code table.

Moreover, CoVE-IO compatible device firmwares must run a Device Security Manager (DSM) to enforce the TDISP-defined security attributes and policies. The DSM must support both full PCIe devices (PFs) and Virtual Functions (VFs) assignment to TVMs.

Enumeration

TDIs PCIe Device Capabilities Registers must have the "TEE-IO Supported" bit in "Device Capabilities Register" set in order for the host to discover their TDISP capability.

Data Object Exchange (DOE)

The TDI implements the optional PCIe Data Object Exchange mechanism. The minimal supported version is 1.0 as defined in the [\[PCIe\]](#) specification.

A CoVE-IO supported device must support the following DOE object types:

Vendor ID	Object Type	Description
0001	00	DOE Discovery
0001	01	CMA/SPDM
0001	02	Secure CMA/SPDM

Secure Protocol and Data Model (SPDM)

RISC-V CoVE-IO compliant hosts use the Security Protocol and Data Model ([\[SPDM\]](#)) protocol to exchange TDISP and IDE Key Management (IDE_KM) messages with physical devices, over DOE mailboxes. CoVE-IO also relies on SPDM for gathering devices' certificates and measurements.

As a consequence, a CoVE-IO supported DSM must support the SPDM specification version 1.2 or above. It must also support sending and receiving SPDM Secured Messages as defined in the [\[SecuredSPDM\]](#) specification version 1.1 or above.

CoVE-IO compatible DSMs must support the following SPDM responder capabilities: CERT_CAP, MEAS_CAP (10b), ENCRYPT_CAP, MAC_CAP, and KEY_EX_CAP.

Moreover, CoVE-IO compatible DSMs must support below algorithms:

1. For **BaseHashAlgo**, one or more of the following ones: TPM_ALG_SHA_256, TPM_ALG_SHA_384.
2. For **MeasurementHashAlgo**, one or more of the following ones: TPM_ALG_SHA_256, TPM_ALG_SHA_384.
3. For **AEAD Cipher Suite**, AES-256-GCM with 16 byte MAC.

CoVE-IO compatible DSMs must support below traditional algorithms:

1. For **BaseAsymAlgo**, one or more of the following ones: TPM_ALG_RSASSA_3072, TPM_ALG_ECDSA_ECC_NIST_P256, TPM_ALG_ECDSA_ECC_NIST_P384.
2. For **DHE Group**, one or more of the following ones: secp256r1, secp384r1.

If CoVE-IO compatible DSMs support SPDM version 1.4 or above, it must support below PQC algorithms, as described in [\[CMAPQC\]](#):

1. For **PqcAsymAlgo**, ML-DSA-87.
2. For **KEMAlg**, ML-KEM-1024.

CoVE-IO compatible DSMs may support following combinations during PQC transition phase:

1. Both traditional algorithms and PQC algorithms
2. Traditional algorithms only
3. PQC algorithms only

Integrity and Data Exchange (IDE)

The data that flows between a TDI and a TVM must be confidential and both integrity and replay attack protected. PCIe IDE provides this protection for all Transport Layer Packets (TLPs) moving between those two endpoints. As a consequence, CoVE-IO compatible devices must implement IDE and expose this PCIe capability defined in the [\[PCIe\]](#) specification.

Selective Streams

As all PCIe switches are excluded from a TVM's TCB, CoVE-IO compliant hosts' TSMs exclusively establish selective IDE streams between the host PCIe Root Port and TEE-IO devices.

Key Management

To fully support IDE, CoVE-IO compatible devices must implement the IDE Key Management (IDE_KM)

protocol. IDE_KM messages are initiated by the TSM and sent over SPDm to the DSM.

CoVE-IO compatible devices must keep track of the IDE key invocation field, which is in bit 63:0 of the IDE sub-stream specific AES-GCM initialization vector (IV). If the IDE key invocation field overflows, the IDE stream must enter Insecure state. Before overflowing, the device may notify the host to let the host software perform the IDE_KM defined key refresh process.

3.2. Host

In order to support TEE-IO devices with the above described capabilities, a CoVE implementation must meet software and hardware requirements, as described in the next sections.

3.2.1. Ownership

In a CoVE-IO context, the hosting Supervisor Domain manager is the sole platform resource's owner. In particular, the physical devices to which TDIs belong are owned and managed by the host, typically through the device PF. Host software stack components, e.g. the VMM, assign, unassign, and expose TDIs resp. to, from, and to TVMs.

3.2.2. Platform Hardware Components

PCIe

All CoVE-IO-compliant devices in a CoVE platform must be connected to the CoVE platform through a PCIe Root Port (RP) that is part of a PCIe Root Complex (RC).

A CoVE-IO PCIe link can be direct or go via any topology or PCIe switches and bridges. Since those are excluded from a TVM's TCB, only selective IDE streams must be used between a CoVE-IO device and its corresponding PCIe Root Port.

The TSM establishes one single selective IDE stream for each physical device from which TDIs may be attached to TVMs. All TDIs within a CoVE-IO device share the same IDE stream.

For a given selective IDE stream, the TSM generates, owns, and distributes the IDE keys to both the CoVE-IO device and its upstream PCIe Root Port.

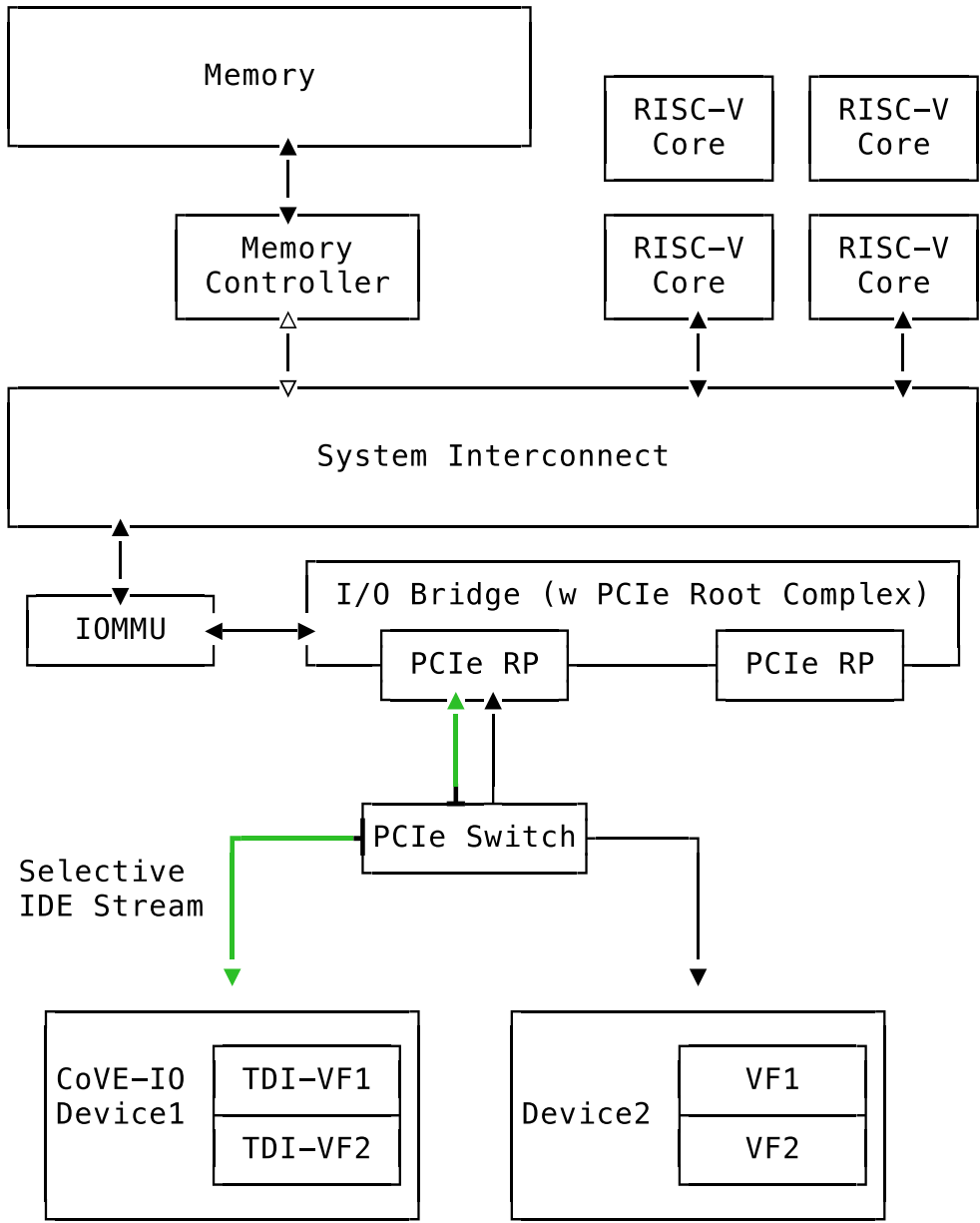


Figure 2. CoVE-IO PCIe Topology

IO Translation Agent

CoVE-IO-compliant platforms must have at least one IO translation agent, also known as an IOMMU. Platform IOMMUs must follow the RISC-V [\[IOMMU\]](#) architecture specification v1.0.

In order to protect direct access to confidential memory, a CoVE-IO device must be attached to a PCIe Root Port that is bound to a platform IOMMU. All inbound traffic from a TDI must then be translated by the upstream IOMMU.

As Supervisor Domain Security Managers (SDSM), TSMs are responsible for setting TDI-specific DMA mappings, MSI page tables, and also for configuring platform IOMMUs own MSIs. As a consequence, IOMMU instances on a CoVE-IO platform may be directly and exclusively tied to a TSM controlled supervisor domain, allowing the TSM to manage address translation and protection for DMA that originates from devices under its control.

Each IOMMU offers a memory-mapped Register Programming Interface (RPI) that the associated TSM uses to configure and control the IOMMU. The RDSM, through the [\[Smmpt\]](#) extension, enforces that this interface is access-restricted to the TSM the IOMMU is assigned to. This guarantees that a TSM currently

running on a RISC-V hart has exclusive access to the physical address of its IOMMU RPI.

On platforms supporting the [\[Smmpt\]](#) and [\[IO-MPT\]](#) extensions, any PCIe device (regardless of its TEE-IO compliance) can be assigned to any Supervisor Domain, as long as this domain has RDSM-enforced exclusive access to an IOMMU instance RPI.

CoVE-IO Manifest

The TSM must be provisioned with a trusted piece of data describing the required CoVE-IO platform components. The Root-of-Trust provides the TSM with a CoVE-IO manifest containing the following pieces of information:

Trust anchor

A list of root certificates that the TSM uses to verify DSM certificates received through SPDML.

IOMMUs

For each IOMMU present in the platform:

- The IOMMU RPI MMIO base address. This is used as the IOMMU identifier.

PCIe Root Ports

For each PCIe Root Ports present in the platform:

- A PCIe Segment:Bus:Device:Function identifier.
- The IOMMU identifier the RP is bound to.
- The list of all MMIO ranges routed through that RP.
- The RP ECAM base address.
- All downstream PCIe Endpoints linked to that RP, identified by their PCIe RID (i.e. the device PCIe Bus:Device:Function triplet).

TODO: More precise CoVE-IO manifest format.

3.2.3. Software

Host

To support extending TVMs with CoVE-IO devices, the untrusted domain software stack must:

- Implement the [\[CoVE\]](#) Host Extension (**COVH**).
- Support the RISC-V [\[IOMMU\]](#) programming interface with an IOMMU driver.
- Implement the CoVE-IO host [COVH](#) ABI.

TSM

The trusted Domain Security Manager, i.e. the TSM, is the trusted intermediary between the untrusted domain and the TVM. To allow for securely assigning TDIs into TVMs, it must:

- Support the [\[CoVE\]](#) Host Extension (**COVH**).
- Implement the [\[CoVE\]](#) Guest Extension (**COVH** and **COVG**).

- Support the RISC-V [\[IOMMU\]](#) programming interface with an IOMMU driver.
- Support the CoVE-IO host ABI, as described in Chapter 9 of this document:
 - Implement the SPDM requester protocol and flows.
 - Implement the TDISP requester protocol and flows.
 - Implement the PCIe IDE Key Management protocol.
- Implement the CoVE-IO guest [COVG](#) ABI.

RDSM

The Root Domain Security Manager (RDSM) is an M-mode component that retains access to the entire physical address space and all platform resources.

To support CoVE and CoVE-IO, the RDSM must implement the Supervisor Domain Access Protection privileged architecture extension (see [\[Smmmtt\]](#)). This extension provides the hardware enforcement mechanism necessary for isolating security domains.

Additionally, the RDSM must implement the CoVE-IO TSM Extension ([COVT](#)) ABI. This interface enables the TSM to request privileged I/O operations, such as programming PCIe IDE keys or configuring Root Ports, which typically require M-mode privileges.

3.3. Guest

A TVM guest must verify and explicitly accept any TDI into their TCBs. The TSM prevents TDIs from directly accessing the TVM confidential memory and prevents the TVM from doing memory mapped I/O with TDIs, unless the TVM guest accepts the TDI.

By implementing the CoVE-IO guest ABI, the TSM allows for a TVM guest to verify the trustworthiness of an assigned TDI. The TVM also uses the same ABI to notify the TSM about its TDI acceptance decision.

The TDI verification process from the TVM guest not only requires support from the TSM through the CoVE-IO guest ABI but may also include running local or remote attestation of the physical device the assigned TDI belongs to. In order to minimize the TVM guest software stack changes needed to support the CoVE-IO TDI verification, attestation, and acceptance flows, the CoVE-IO guest must run a Trusted Device Manager (TDM) as a separate TVM guest process. Although the TDM can be architected in a TEE-agnostic fashion, it must support the CoVE-IO guest ABI.

Chapter 4. Security Model

The CoVE-IO security model is built on the following assumptions:

- The host platform physical devices are owned by untrusted domain software components (e.g. the host VMM or the hypervisor) that are not part of any TVM TCB and are thus untrusted by TVMs.
- The TSM is the security policy enforcer.
- Only the TVM owner can assess of a TDI trustworthiness. Based on that assessment, it explicitly accepts or rejects a TDI into its TCB.
- A TDI may access a TVM confidential memory through DMA only when all the following conditions are met:
 - The TVM owner has explicitly allowed the TDI to access its confidential memory by accepting it.
 - The TDI is exclusively assigned to the TVM, i.e. it must not be shared with other TVMs or any host software component. This is enforced by the TSM.
- A TVM may access a TDI trusted MMIO space only when all the following conditions are met:
 - The TVM owner has explicitly accepted the TDI.
 - The TDI is exclusively assigned to the TVM, i.e. it must not be shared with other TVMs or any host software component. This is enforced by the TSM.
- Until a TDI is accepted by the TVM:
 - The TDI is not allowed to DMA into the TVM confidential memory.
 - Trusted MMIO access to the TDI is blocked by TSM.
 - Untrusted MMIO access to the TDI may still be allowed by the VMM for ordinary VMs (if and only if the TDI is in the TDISP unlocked state).

By means of the CoVE-IO guest ABI, TVMs are required to explicitly accept TDIs into their TCBs. Accepting or rejecting a TDI is a TVM specific decision, based on TVM specific set of verification policies and criteria. A TVM accepting a TDI does not imply that other TVMs on the same host platform would accept it as well.

Shareable CoVE-IO-compliant devices may expose multiple TDIs, assigned to different TVMs. It is the DSM responsibility to guarantee isolation between all assigned TDIs, on a per-TVM basis.

Each TDI must be exclusively assigned to no more than one TVM. However, a single TVM can simultaneously be assigned several TDIs, irrespective of whether they originate from the same physical device or not.

As a TVM TCB does not include PCIe switches and bridges, a selective PCIe IDE stream must be setup to guarantee end-to-end confidentiality and integrity protection between a TVM and a TDI. The host VMM is responsible for reserving a single selective IDE stream per physical device from which one or more TDIs are assigned to TVMs. That single PCIe IDE stream is then shared by all TDIs originating from the same corresponding physical device. Since the host VMM is not in the TCB, its role in the stream setup is limited to selecting an available stream ID and configuring the device IDE capability. The selective IDE stream keys, for both link endpoints, are managed and configured by TCB elements (the TSM, DSM, and RDSM).

The main security objective of the CoVE-IO security model is to protect a TVM's confidential data integrity and confidentiality while TDIs are assigned to it. At the same time, availability of those assigned TDIs is out of this model scope as e.g. the host VMM could remove them from the TVM at any time it sees fit. Either the DSM or the TDI itself must clear and wipe all TVM confidential data in the TDI before the host software stack can fully reclaim an assigned TDI.

Chapter 5. Threat Model

5.1. Assets

The CoVE-IO security model aims at protecting the following assets:

1. The TVM confidential data, which includes its confidential main memory, code and execution state. A TVM execution state includes both its CPUs micro-architectural states and its assigned TDIs states.
2. An assigned TDI trusted I/O space that is mapped into the TVM address space, also known as a TDI trusted MMIO.

5.1.1. Security Objectives

The CoVE-IO security model objectives is to protect the above-described TVM assets **confidentiality** and **integrity** from components outside of the TCB.

Availability of these assets is out of the CoVE-IO security model scope.

5.2. Adversary Model

The CoVE-IO security model aims at protecting the above-described TVM assets from the following adversaries:

- *Startup Software adversary*: This includes host system software executing in early boot phases of the system in M-mode, including M-mode firmware, memory configuration code, device option ROM that can access system memory, CPU registers, IOMMU(s), IO devices and platform configuration registers (e.g., address range decoders, SoC fabric configuration, etc.).
- *Privileged host software adversary*: This includes host software components executing in S/HS mode like the kernel, VMM, host device driver, etc. As the system resource owner but also the TVMs lifecycle manager, those components can access and control all devices on the system.
- *Unprivileged host software adversary*: This includes host software components executing in U mode like e.g. the userspace parts of the host VMM.
- *Privileged guest software adversary*: This includes guest software components executing in VS mode like e.g. the guest kernel, the guest device driver.
- *Unprivileged guest software adversary*: This includes guest software components executing in VU mode like e.g. the guest application workload.
- *Device firmware adversary*: This includes any firmware driving a device within the system's PCIe topology. It is the firmware running on the device itself, other than DSM. As PCIe transaction generators those device firmware components can gain direct access to a TVM confidential memory.
- *Simple hardware adversary*: This includes adversaries that are able to probe visible buses on the motherboard, use JTAG based debuggers, power cycle the system, subject the system to thermal radiation.
- *Advanced hardware adversary*: This includes adversaries that, in addition to the simple hardware adversary capabilities, can also probe high speed buses, place interposers on visible buses, glitch clocks and voltage rails.

5.3. Threats

5.3.1. CoVE-IO-T001 - Trusted MMIO Malicious Access

Table 1. CoVE-IO-T001

Asset	Threat	Adversary	Scope	Result
TVM confidential data	Tamper and Disclosure	Privileged host software	In scope	Host component reads TVM confidential data
Description				
A privileged host software component programs a TDI that is assigned to a TVM. By accessing the device MMIO space, the host component can program direct memory access destination addresses to either its own address space or unintended parts of the TVM address space. Device generated data intended to be copied to the TVM confidential memory is respectively accessed by the host component instead or redirected to unintended parts of the TVM address space.				
Mitigations				
The CoVE-IO-T001 threat can be addressed by preventing untrusted domain software components from accessing an assigned TDI, as follows: • With TEE-I/O, a PCIe root port generates TLPs with the T-bit set only if the MMIO access originates from the trusted domain. Untrusted domain MMIO accesses must have the T-bit cleared. • A TDI is assigned to a TVM when the TVM accepts it into its TCB, by notifying the TSM about it. • The TEE-I/O DSM enforces that: • Before it is assigned to a TVM, a TDI must not directly access the TVM confidential memory. • Once assigned to a TVM, a TDI is in either the LOCKED or RUN TDISP state. • In both the LOCKED and RUN TDISP state, a TDI trusted MMIO space can only be accessed by a trusted domain generated TLP (T-bit set), through the TDI bound PCIe selective IDE stream.				

5.3.2. CoVE-IO-T002 - Trusted MMIO Remapping

Table 2. CoVE-IO-T002

Asset	Threat	Adversary	Scope	Result
Device trusted MMIO	Tamper	Privileged host software	In scope	TVM programs a TDI that is unassigned to it
Description				
A privileged host software component remaps a TVM assigned TDI MMIO guest physical address to an unassigned TDI MMIO host physical address. The TVM programs a different TDI than the one that is assigned to it.				
Mitigations				

Asset	Threat	Adversary	Scope	Result
The CoVE-IO-T002 threat can be addressed as follows:				
• The TSM maintains second stage page tables (from trusted domain physical addresses to untrusted host domain physical addresses) in confidential memory. • The untrusted domain software component must not set the second stage mappings for the TDI trusted MMIO. It can requests the TSM to do so on its behalf, through the CoVE-IO host ABI. • The TSM must not enable Trusted MMIO mappings for an assigned TDI until the TVM accepts it. • The TVM receives the TDI device interface report through TDISP, via the the TSM CoVE-IO guest ABI. This report is trusted by the TVM and contains the trusted MMIO ranges and order in which they must be mapped to the TVM address space. • The TVM must explicitly accept the reported MMIO ranges, and the TSM must not enable them until they are accepted by the TVM.				

5.3.3. CoVE-IO-T003 - Trusted MMIO PCIe Redirection

Table 3. CoVE-IO-T003

Asset	Threat	Adversary	Scope	Result
Device Trusted MMIO	Tamper	Privileged host software	In scope	TVM accesses an unassigned TDI trusted MMIO space
Description				
A privileged host software component configures PCIe switches to redirect (or drop) MMIO accesses from the TVM to one of its assigned TDIs. The host software component can trick the TVM into tampering with an untrusted device or an unassigned TDI MMIO.				
Mitigations				
The CoVE-IO-T003 threat can be addressed as follows: • PCIe switches must not be included in the TVM trust boundary. This is achieved by only allowing PCIe selective IDE streams to be established between a physical device and the untrusted host domain. • Although the VMM can tamper with the device IDE extended capabilities, the PCIe root port IDE settings must only be available to a TVM TCB component, either the TSM or the platform Root-of-Trust.				

5.3.4. CoVE-IO-T004 - Trusted MMIO PCIe Pre-Configuration

Table 4. CoVE-IO-T004

Asset	Threat	Adversary	Scope	Result
TVM confidential data	Tamper and Disclosure	Privileged or unprivileged host software	In scope	Guest software reads and writes resp. from and to another TVM confidential memory
Description				
The VMM maliciously pre-configures a TDI trusted MMIO and assigns it to a TVM. If either the TVM accepts the TDI as-is into its TCB, or the TDI is made operational before the TVM accepts it, the TDI can now access or tamper with the TVM confidential data on behalf of the host software component.				
Mitigations				

Asset	Threat	Adversary	Scope	Result
The CoVE-IO-T004 threat can be addressed as follows:				
• The TVM must explicitly accept the reported trusted MMIO ranges before any operation. • The TSM must not enable Trusted MMIO mappings for an assigned TDI until the TVM accepts it.				

5.3.5. CoVE-IO-T005 - Trusted MMIO Unauthorized Access

Table 5. CoVE-IO-T005

Asset	Threat	Adversary	Scope	Result
Device trusted MMIO	Tamper	Privileged host software	In scope	TVM accesses an unassigned TDI trusted MMIO space
Description				
A privileged host software component maps a TDI trusted MMIO space into TVM1 as part of the TDI assignment. Then it unassigns the TDI from TVM1 and assigns it to TVM2, without unmapping the TDI trusted MMIO space from TVM1. TVM1 can tamper with a TDI trusted MMIO while it is not assigned to it.				
Mitigations				
The CoVE-IO-T005 threat can be addressed as follows: • The TSM must ensure a TDI is assigned to only one TVM. Once the TDI is assigned, it cannot be assigned to the other TVM. The TDI can be assigned to the other one, only after it is stoped. • The TSM must ensure Trusted MMIO is mapped to only one TVM. Once the MMIO is mapped, it cannot be mapped to the other TVM. The MMIO can be mapped to the other one, only after it is unmapped.				

5.3.6. CoVE-IO-T006 - PCIe Link Man-In-The-Middle

Table 6. CoVE-IO-T006

Asset	Threat	Adversary	Scope	Result
TVM confidential data	Tamper and Disclosure	Advanced hardware	In scope	A hardware adversary probes or places an interposer on the PCIe physical link between a TVM and its assigned TDI
Description				
A skilled hardware adversary with system physical access probes or places an interposer in the PCIe physical link. It can then eavesdrop, replay or event tamper with a TVM confidential data.				
Mitigations				
The CoVE-IO-T006 threat can be addressed as follows: • A PCIe root port only accepts the DMA request to a trusted domain with IDE TLPs with the T-bit set. • The device only accepts the trusted MMIO request to a TDI with IDE TLPs with the T-bit set.				

5.3.7. CoVE-IO-T007 - PCIe ID Spoofing

Table 7. CoVE-IO-T007

Asset	Threat	Adversary	Scope	Result
TVM confidential data	Tamper and Disclosure	Device firmware	In scope	Device firmware reads and writes from and to a TVM confidential memory
Description				
A device firmware spoofs a PCIe Requester ID (RID) to generate PCIe packets with an existing, assigned TDI RID and get direct memory access to the corresponding TVM confidential memory.				
Mitigations				
The CoVE-IO-T007 threat can be addressed as follows: • A PCIe root port must only accept the IDE TLP with T-bit set to access the TVM confidential memory. • A PCIe root port must check IDE TLP source RID with the IDE stream RID and reject the TLP if there is RID mismatch.				

5.3.8. CoVE-IO-T008 - Confused Deputy DMA Remapping

Table 8. CoVE-IO-T008

Asset	Threat	Adversary	Scope	Result
TVM confidential data	Tamper and Disclosure	Privileged guest software	In scope	Guest software reads and writes resp. from and to another TVM confidential memory
Description				
TVM1 and TVM2 are assigned resp. TDI1 and TDI2. TDI1 and TDI2 belong to the same physical device. TVM1 programs TDI1 with TVM2's address space. TVM2 confidential memory is accessed by an unassigned TDI.				
Mitigations				
The CoVE-IO-T008 threat can be addressed as follows: • The TSM must guarantee the DMA translation table for one TDI can only access the corresponding TVM. • The TSM must guarantee the invalidation of all translation caches associated with the DMA translation table if there is change, including but not limited to CPU TLB, IOMMU TLB and device TLB.				

5.3.9. CoVE-IO-T009 - DMA Remapping

Table 9. CoVE-IO-T009

Asset	Threat	Adversary	Scope	Result
TVM confidential data	Tamper and Disclosure	Privileged host software	In scope	Host software reads and writes from and to a TVM confidential memory
Description				
The privileged host software component manipulates an assigned TDI guest physical address (GPA) to host physical address (HPA) mappings. The TDI direct memory access to and from the TVM confidential data is then redirected to the host software component address space, allowing it to eavesdrop or tamper with the TVM confidential data.				
Mitigations				

Asset	Threat	Adversary	Scope	Result
The CoVE-IO-T009 threat can be addressed as follows: • The RDSM must guarantee that a DMA transaction from one TDI is translated by an IOMMU instance controlled by a TSM that manages the TVM to which the TDI is bound to. • The TSM must guarantee that the DMA translation table for one TDI can only access the corresponding TVM. • The DSM must guarantee that the DMA request uses IDE TLP with T-bit set.				

5.3.10. CoVE-IO-T010 - DMA Remapping

Table 10. CoVE-IO-T010

Asset	Threat	Adversary	Scope	Result
TVM confidential data	Tamper	Privileged host software	In scope	TDI writes into unintended portions of a TVM confidential memory
Description				
The privileged host software component manipulates the guest physical address (GPA) to host physical address (HPA) mappings to create inconsistencies between the TVM and its assigned TDI mappings for the same GPA ranges. The TDI writes physical addresses that are different than the ones the TVM programmed it with, and tampers the TVM confidential memory. Moreover, the TVM memory reads from the intended GPA return results that are inconsistent with the actual device operation.				
Mitigations				
The CoVE-IO-T010 threat can be addressed as follows: • The RDSM must guarantee that a DMA transaction from one TDI is translated by an IOMMU instance controlled by a TSM that manages the TVM to which the TDI is bound to. • The TSM must guarantee that the DMA translation table for a TDI under its control is consistent with the G-stage tables for the TVM the TDI is bound to. • The TVM must accept the DMA translation table explicitly. • The TSM must not enable DMA translation table until the TVM accepts the TDI.				

5.4. Requirements

List CoVE-IO security requirements to address the threat model.

Chapter 6. Architectural Overview

CoVE-IO is made of multiple hardware and software components.

Figure 1 below illustrates the overall CoVE-IO architectural components and how they interact together:

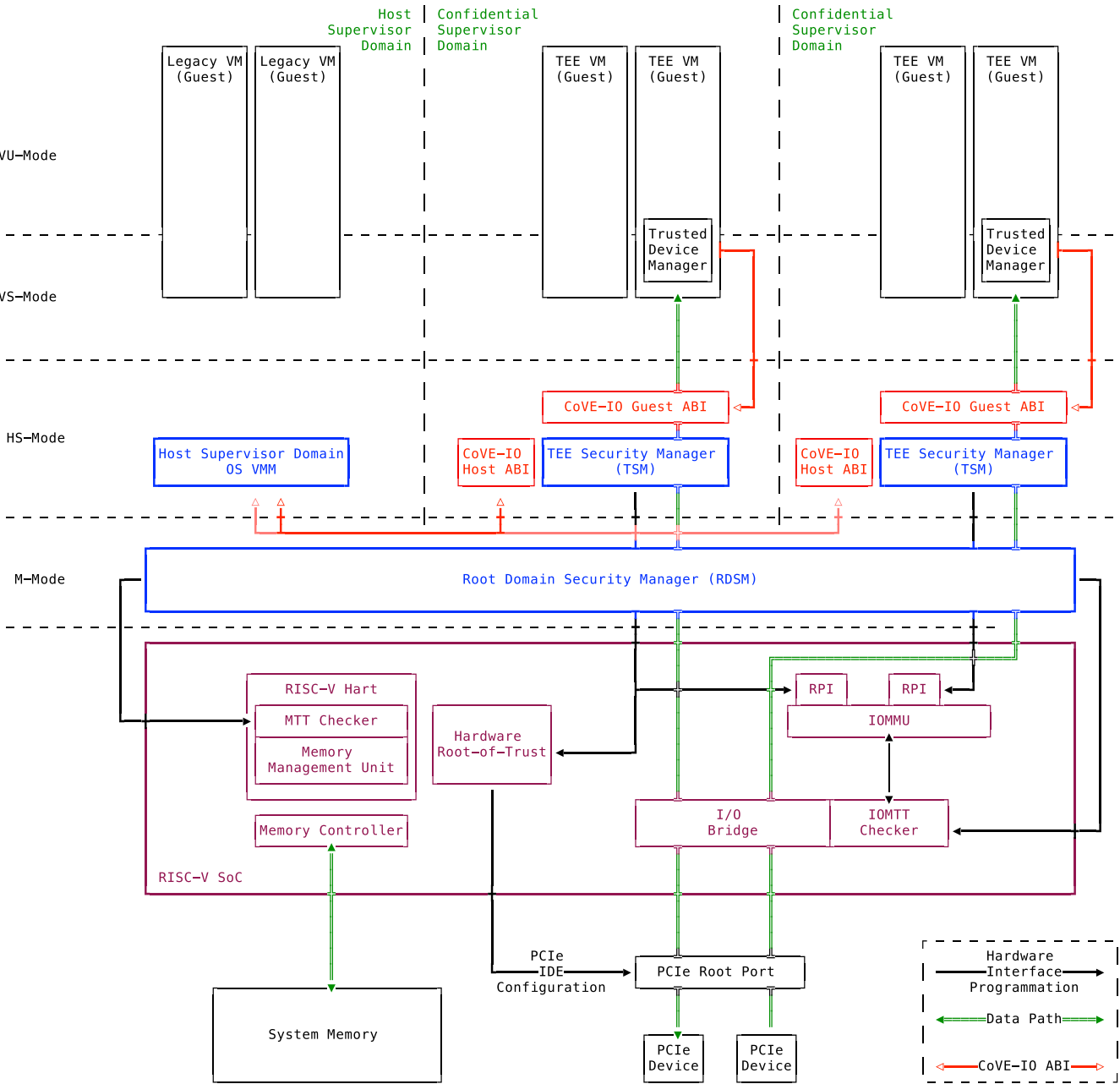


Figure 3. CoVE-IO High Level Architecture

The TEE Security Manager (TSM) orchestrates and manages TEE Virtual Machines (TVM), as defined by the [CoVE] specification. The TSM operates in HS-Mode and is the confidential Supervisor Domain manager. It uses the [Smmpt] specified Memory Protection Table (MPT) to enforce TVM memory isolation from the host Supervisor Domain.

As the confidential Supervisor Domain manager, the TSM provides the **COVH** ABI for the host OS/VMM to create and destroy TVMs, and donate or reclaim confidential memory from them. TVMs can interact with the TSM through the **COVG** ABI.

CoVE-IO extends both the **COVH** and **COVG** ABIs for bringing TEE-IO capable devices into the trust boundary of TVMs. These extensions allow for configuring and registering PCIe root ports and then binding physical

devices interfaces (TDI) and TVMs together.

With the CoVE-IO ABIs and flows, TDIs can access TVM confidential memory directly through DMA. Based on the TVM configuration, the DMA accessible confidential memory could be all TVM memory or a subset of the TVM memory. At least one platform IOMMU instance must be exclusively tied to each supervisor domains managing TVMs to which TDIs can be bound to. CoVE-IO uses the [\[Smmpt\]](#) and [\[IO-MPT\]](#) extensions, together with the IOMMU instance memory-mapped register programming interface (**RPI** in figure 1) to grant TDIs with direct access to their bound TVM confidential memory and isolate it from DMA originating from any unbound TDI.

With the CoVE-IO model, the TVM ultimately holds the decision to accept or reject a bound TDI into or from its TCB. Access to a TVM confidential memory from a TDI is disabled until a TVM notifies the TSM that it trusts the TDI and its configuration. The TVM communicates this decision after verifying the TDI, which generally includes attesting to the physical device the TDI belongs to. The device attestation process is specific to each TVM workload configuration, but it may be handled by the Trusted Device Manager (TDM).

To ensure the confidentiality and integrity of TVM data, PCIe IDE streams are established between the PCIe Root Port and its bound TDI. Under the CoVE-IO framework, the TSM is responsible for generating and configuring the necessary IDE keys. Specifically, the TSM configures the PCIe physical device using the PCIe IDE Key Management (**IDE_KM**) protocol. However, because PCIe Root Port key management is platform-specific, the TSM delegates those operations to the platform's RDSM.

Chapter 7. Theory of Operations

The CoVE-IO specification extends the CoVE host and guest SBI extensions to allow TVMs to establish trust with TEE-IO devices, and then use and interact with those devices. Untrusted Supervisor Domain components are responsible for assigning TEE-IO devices to TVMs, and also for supporting the TVM acceptance or rejection of the assigned devices. The TSM, on the other hand, establishes and maintains the secure physical and logical links between TVMs and their assigned devices.

The following sections describe the functionality of the TSM-provided CoVE-IO extensions to support trusted I/O on CoVE-enabled platforms.

7.1. Platform Management

7.1.1. IOMMU Assignment

For CoVE-IO, a TSM relies on being granted exclusive control over one IOMMU instance on the platform. An IOMMU instance refers to any instantiation of a RISC-V IOMMU Register Programming Interface (RPI), which could range from an actual IOMMU instance to an MPT-enforced, memory-mapped access to a physically partitioned IOMMU.

This exclusively assigned IOMMU RPI allows the TSM to enforce the integrity of address translations and protection from DMA into confidential memory, as well as interrupts originating from TDIs assigned to TVMs under the TSM control.

When creating a Supervisor Domain (SD), the RDSM can optionally establish exclusive access to an IOMMU RPI for the SD TSM, prior to loading and scheduling said TSM. The RDSM creates [\[Smmpt\]](#) mappings to assign the IOMMU RPI to the SD and isolate it from any other domain, including the hosting SD.

When the hosting SD requests a TDI to be bound to a TVM, the RDSM programs the I/O-MPT Supervisor Domain Classifier (SDCL) to map DMA requests originating from the TDI to the TVM's SD and its IOMMU. Concurrently, the TSM programs the IOMMU RPI with the necessary IOVA translations and interrupt remappings. However, to maintain security, the TSM must not enable these mappings until the TVM has successfully verified and accepted the TDI, as detailed in the [TDI acceptance](#) section.

7.1.2. PCIe Root Port Management

When the data link between the TDI and the TVM must be secured, trusted I/O relies on the PCIe IDE protocol. IDE provides confidentiality and integrity protection for TLPs received and transmitted between the physical device the TDI belongs to and its assigned PCIe root port (RP). Both endpoints (The RP and the physical device) must be configured with the same encryption keys through the IDE key management protocols and have their IDE PCIe extended capability configured as well.

As described in the threat model and requirement sections, the following trusted I/O rules apply:

- As PCIe switches are outside of the TVM TCB, only selective IDE streams are used to protect the PCIe link between the TVM and the TDI.
- A single selective IDE stream is established between the physical device and its Root Port. All TDIs from the physical device share this single IDE stream.
- The TSM that establishes the SPDm connection with a physical device must generate, own and distribute the IDE stream keys to both the physical device and the RP.

- A TDI must not be bound to a TVM if the TSM managing the TVM does not own the IDE keys for the TDI's physical device.
- As the owner of the PCIe configuration space for all RPs on the platform, the RDSM configures all IDE streams extended capability on behalf of Supervisor Domain Managers (host and TSMs).

PCIe Root Port IDE Key Programming

During [Device Connection](#) establishment with a TEE-IO capable device, the TSM generates and provisions IDE keys to two locations: the device itself and its upstream PCIe Root Port (RP).

While provisioning the device is done using the standard PCIe IDE Key Management Protocol (IDE_KM), provisioning the Root Port is a platform-specific operation. This operation is delegated to the Root Domain Security Manager (RDSM), which the TSM accesses via the **COVT** SBI's PCIe Root Port IDE management functions.

PCIe Root Port IDE Extended Capability

Since the PCIe configuration space occupies exactly one 4KB page and MPT access controls operate at page granularity, the RDSM (as the platform security enforcer) must retain exclusive ownership of the entire configuration space for all RPs. Consequently, it enforces that Supervisor Domain Managers configure the IDE Extended Capability for their owned streams via the **COVT** SBI `sbi_covt_program_rp_ide_stream` function.

When requested to establish a [connection](#) with a TEE-IO capable device, the TSM receives the IDE Extended Capability configuration for the allocated IDE stream from the Host Supervisor Domain. After successfully establishing the connection, it configures the IDE stream capabilities via the `sbi_covt_program_rp_ide_cap` SBI function.

7.2. Device Initialization

A device may require special initialization before it can act correctly in confidential computing mode. This initialization work should be done by device specific software, also known as device driver. The location of the device driver may depend on how the device is assigned to a TVM.

In single root I/O virtualization (SR-IOV) mode, the host VMM owns the device physical function (PF) and the TVM owns the device virtual function (VF). The TDI is the device VF. The device initialization should happen in the host device driver in the VMM.

In discrete device assignment (DDA) mode, the entire physical device is assigned to a TVM. The TDI is the entire device. The device initialization may happen in the guest device driver in TVM. The host VMM does not include any device driver. In this case, the VMM should ensure that the guest device driver does the initialization before the VMM triggers any common confidential computing flow, such as SPDM session establishment or IDE key management.

With DDA mode, the TVM should be configured so that it automatically initializes the physical device without TSM or VMM intervention. Moreover, the TVM is responsible for notifying the VMM once initialization completes, through some out-of-band transport like e.g. paravirtualized I/O. Both the TVM configuration details and the notification mechanism are out of the CoVE-IO scope.

7.3. SPDM Transport

SPDM is the main transport protocol for securely setting both the physical device IDE keys and also for

driving the TDI through the TDISP state machine.

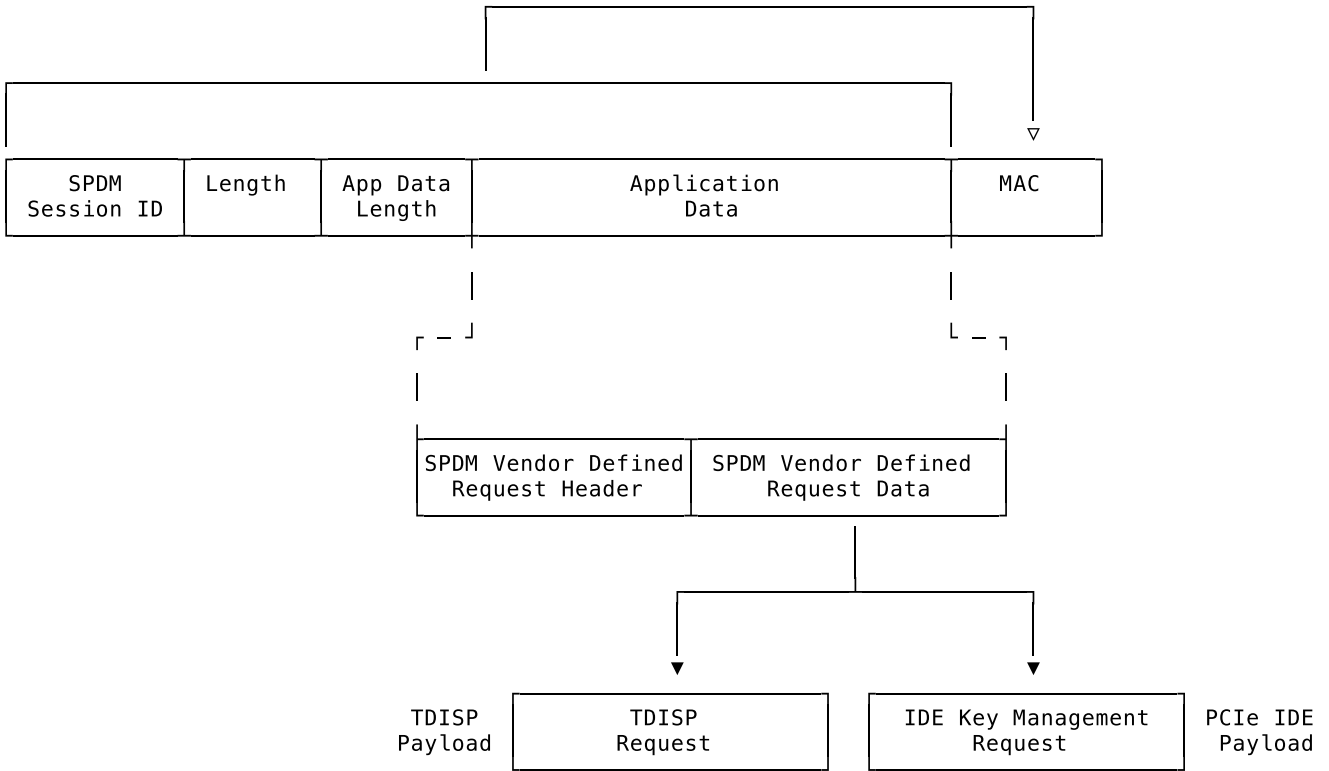


Figure 4. SPDM Message Layout

The TSM is the SPDM requester and it must reach and communicate with DOE mailboxes that are owned by the hosting Supervisor Domain manager. As a consequence, after requesting the TSM to establish an SPDM session with a device (through the `sbi_covh_connect_device()` COVH call), it acts as an untrusted SPDM messages proxy between the TSM and the device DSM.

After a successful call to `sbi_covh_connect_device()`, hosting Supervisor Domain initiated CoVE-IO COVH calls that require SPDM requests to be sent to the device DOE mailbox follows the flow described below:

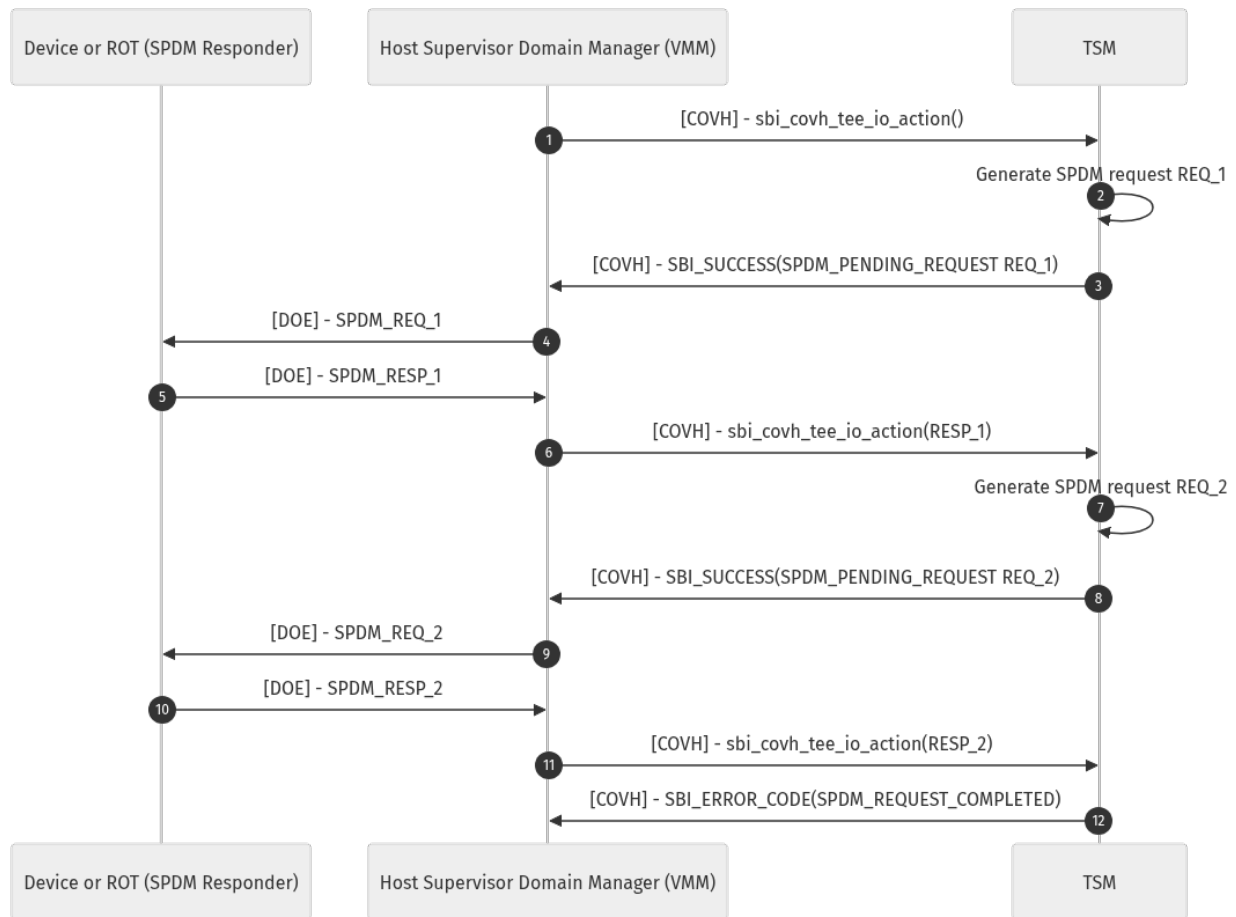


Figure 5. SPDM Flow With CoVE

The TSM generates the SPDM request to support the initial CoVE-IO **COVH** call and copies the request into the per-vcpu shared, non-confidential memory region that the hosting Supervisor Domain and the TSM share as per the CoVE specification. The TSM replies to the hosting Supervisor Domain manager request with the **SBI_SUCCESS** error code and the **SPDM_PENDING_REQUEST (0x1)** value through the **sbiret** structure. The hosting Supervisor Domain manager then sends the pending SPDM request to the device DOE mailbox. It forwards the device SPDM response to the TSM, by copying it to the same SPDM buffer it fetches the SPDM request from and by calling again the same CoVE-IO **COVH** call. This process continues until the initial CoVE-IO call is completed. The TSM then replies to the last **COVH** call with the appropriate error code and the **SPDM_REQUEST_COMPLETED (0x0)** value through the **sbiret** structure.

The TSM only supports one pending SPDM transaction per device, and the CoVE NACL shared memory holds one pending SPDM action buffer per device the TSM is connected to. Each pending SPDM action buffer is structured as described in the following layout:

Table 11. Pending SPDM Transaction Buffer Layout

Offset (bytes)	Field	Length (bytes)	Description
0h	FUNCTION_ID	4	The SBI Function ID this pending SPDM transaction applies to.
4h	DEVICE_ID	4	The PCIe device ID this pending SPDM transaction applies to.
8h	SPDM_PAYLOAD_LENGTH	4	SPDM payload length.
12h	SPDM_PAYLOAD	See SPDM_PAYLOAD_LENGTH	SPDM payload data (e.g. TDISP, IDE_KM).

In this document, for readability reasons, flow definitions that involve SPDm based exchanges are simplified and do not include the above described flow between the TSM, the hosting Supervisor Domain manager and the SPDm responder. In particular, the steps that cover the SPDm request generation from the TSM, the transmission to the host VMM, the transmission to the SPDm responder and finally the path back to the TSM are reduced into 2 steps:

1. TSM sends SPDm request #1
2. TSM receives SPDm response #1

For example the above example would be described through those simplified steps:

1. VMM calls `sbi_covh_tee_io_action()`
2. TSM sends SPDm request #1
3. TSM receives SPDm response #1
4. TSM sends SPDm request #2
5. TSM receives SPDm response #2
6. TSM returns `sbi_covh_tee_io_action()`

7.3.1. Secure SPDm Session

With trusted I/O, the TSM establishes SPDm sessions with the physical devices DSMs. As those sessions are used to exchange IDE keys through the **IDE_KM** protocol while going through the hosting Supervisor Domain manager, they need to be confidentiality and integrity protected. Establishing a secured SPDm session for IDE key management or TDISP operations between the TSM and devices is a mandatory requirement for TVMs to accept TDIs into their TCB.

Any trusted I/O SPDm session is established through the SPDm responder DOE mailbox in the physical device DSM. The mailbox is a resource owned by the hosting Supervisor Domain manager which thus initiates the SPDm session establishment. It acts as an untrusted proxy between the TSM and the DSM by requesting the TSM to generate SPDm requests through the CoVE-IO **COVH** ABI, sending those requests to the DOE mailbox and forwarding the SPDm responses back to the TSM, as described in the SPDm flow section.

7.4. Device Connection

After the IOMMU is registered with the TSM, the hosting Supervisor Domain manager must establish a logical connection with any device from which a TDI could be bound to a TVM. To do so, it must cooperate with the TSM to properly initialize all such physical devices.

The device initialization process aims at establishing secured, integrity-protected control and data planes between the TSM and the DSM running in either the platform ROT or a physical PCIe device. The secured control plane is based on the SPDm protocol and is an encrypted, integrity-protected software session that is used for passing TDISP and IDE_KM messages between the TSM and the DSM. The data place is a hardware session based on the PCIe Integrity and Data Encryption (IDE) specification and is used to secure the PCIe TLPs.

When the hosting Supervisor Domain detects a new TEE-IO capable device, it must go through two device initialization steps:

1. Establish a secured SPDm session between the TSM (The SPDm Requester) and the device DSM (The SPDm responder).

2. Set the PCIe IDE stream up for encrypting the PCIe link.

The CoVE-IO COVH extension supports those two initialization steps through one single function: `sbi_covh_connect_device()`.

When the hosting Supervisor Domain manager calls `sbi_covh_connect_device()`, it requests the TSM to establish an SPDm session with a device, and to set a PCIe IDE link between the device and the TSM.

7.4.1. SPDm Session

The first step for initializing a TEE-IO capable device is to establish a secured SPDm session between the TSM and the device. The secured SPDm sessions will then be used to carry **TDISP** and **IDE_KM** messages, in order to respectively secure the physical link between the device and its PCIe root port, and for binding or unbinding TDIs to or from a TVM.

The hosting Supervisor Domain manager requires the TSM to establish a secured SPDm session with the physical device by calling the `sbi_covh_connect_device()` COVH function.

The TSM establishes a secured SPDm session with the physical device DSM by going through the steps described in the Secured SPDm Session section.

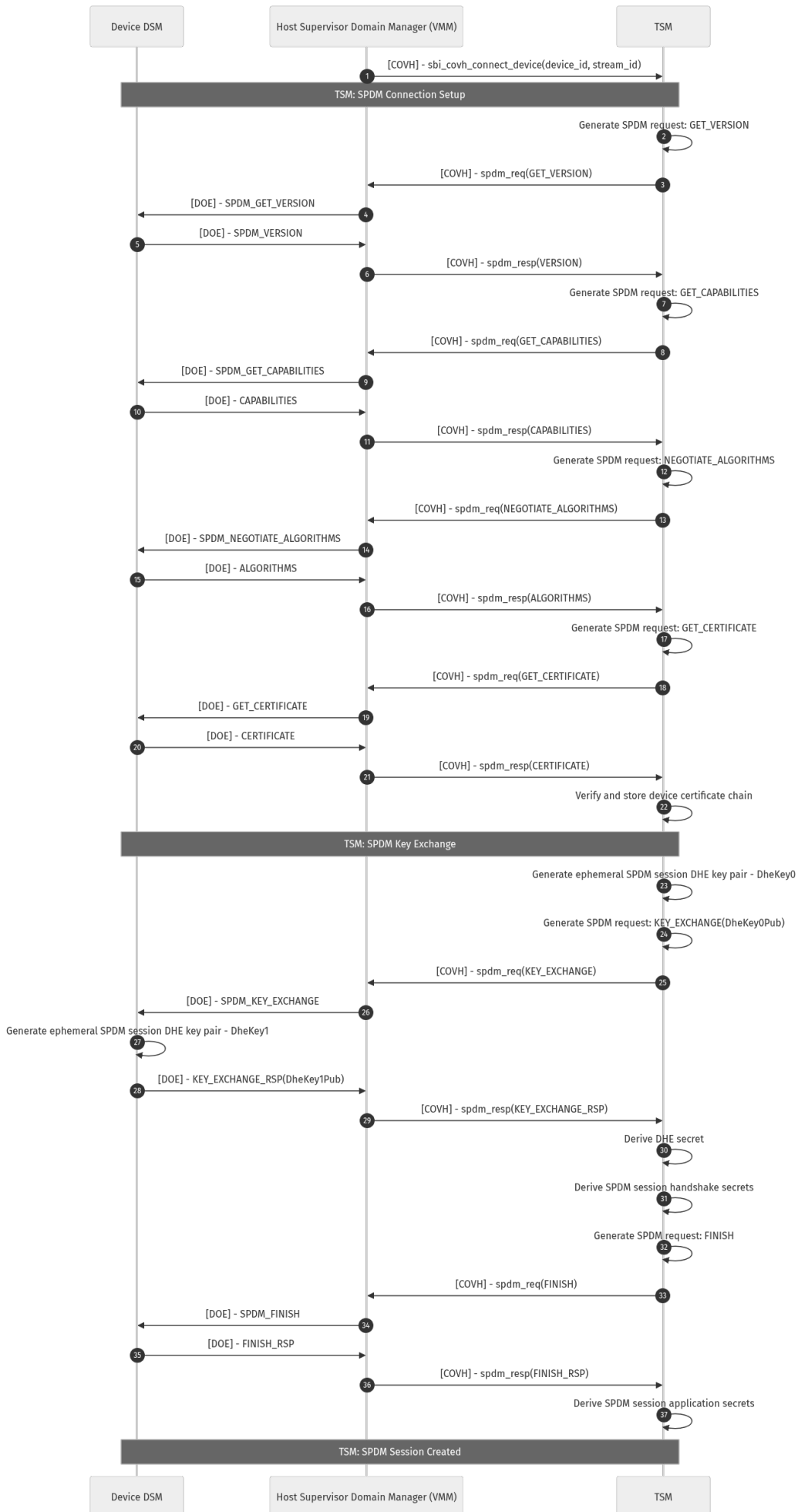


Figure 6. Device Connection - Secured SPDM Session

7.4.2. Device Connection Transcript

The CoVE-IO device connection transcript is a complete, ordered record of all SPDm requests and responses exchanged between the TSM and a device during the connection establishment process. The transcript is a foundational element for subsequent verification and security checks. It includes the messages leading to the **Version-Capabilities-Algorithm (VCA)** construction and the device's certificate chain and measurements.

The CoVE-IO device connection transcript is constructed from raw SPDm messages and must adhere to the following properties:

- Contains only plain text SPDm messages (no encryption).
- Contains full SPDm messages, including digital signatures, to allow for checking message integrity and authenticity.
- Strictly follows the SPDm message exchange order between the Requester (TSM) and the Responder (device).
- Must not include any secret data, such as IDE keys.
- Must not include transport layer padding.
- Must not include **ENCAP** SPDm message headers, or **CHUNK** headers. Large messages transmitted in chunks must be reconstructed before being added to the transcript.
- May include optional TDISP messages exchanged by the TSM (e.g. **TDISP_GET_VERSION** or **TDISP_GET_CAPABILITIES**).

Below is an example of all messages composing a device connection transcript:

```
[GET_VERSION][VERSION]
[GET_CAPABILITIES][CAPABILITIES]
[NEGOTIATE_ALGORITHMS][ALGORITHMS]
[GET_CERTIFICATE][CERTIFICATE]
[KEY_EXCHANGE][KEY_EXCHANGE_RSP]
[FINISH][FINISH_RSP]
[GET_MEASUREMENTS][MEASUREMENTS]
[TDISP_GET_VERSION][TDISP_VERSION]
[TDISP_GET_CAPABILITIES][TDISP_CAPABILITIES]
```

Once the connection is established, the TSM returns the complete connection transcript to the hosting Supervisor Domain manager via the **sbi_covh_connect_device()** ABI. The TSM retains and maintains a map of the connection transcript digests for all connected devices.

TVM Verification Process

As part of the [TDI verification process](#), a TVM can request device connection-related data from the TSM to verify the connected device's trustworthiness or the integrity of the secure SPDm session.

The TSM handles these requests sent by the TVM via the **sbi_covg_get_device_connection_transcript()** ABI:

1. Retrieving the full device connection transcript from the host, through the **sbi_covh_get_device_connection_transcript()** ABI.

2. Verifying the transcript's digest against the its internal map to ensure integrity.
3. Returning the requested data based on the `transcript_element_id` value. This could be the entire transcript or a specific element from it.



*When receiving the full transcript, TVM verifiers must inspect all **MEASUREMENT** messages and may need to reconstruct device certificates that were potentially delivered across multiple **CERTIFICATE** messages.*

7.4.3. IDE Link

The SPDMM session is a software link between the TSM and the DSM, secured after both entities go through a DHE key exchange over the untrusted host supervisor domain manager proxy. SPDMM is used as a control link for configuring the rest of the device and then running the TDI binding flows.

The last part of the device connection process is about securing the data link between the TSM and the device, and that must be done through the IDE Key Management protocol. Here again, the hosting Supervisor Domain manager implicitly initiates the PCI IDE link setup by calling the `sbi_covh_connect_device()` COVH function, and relies on the TSM to generate and send IDE KM messages over SPDMM.

The TSM is responsible for:

1. Configuring the PCIe root port IDE Extended Capability.
2. Generating the IDE keys for all sub-streams for a given stream ID.
3. Setting the PCIe root port IDE keys for a given stream ID, through the `COVT` PCIe Root Port Management functions.
4. Refreshing the PCIe root port IDE keys for a given stream ID, upon the host Supervisor Domain manager request.
5. Generating all IDE KM requests and encapsulating them into SPDMM messages.
6. Setting the PCIe device IDE keys through IDE KM requests.

The hosting Supervisor Domain manager is responsible for:

1. Generating and managing system wide PCIe stream IDs.
2. Setting the device PCIe device IDE PCI Extended Capability.
3. Programing the PCIe switch between the device and PCIe root port, such as IDE Control Register - Flow-Through IDE Stream Enabled.
4. Initiating the IDE link setup.
5. Initiating IDE keys refresh.

The IDE link initial setup must go through the following steps:

1. The hosting Supervisor Domain manager finds an available stream ID and configures the device IDE Extended Capability accordingly.
2. The hosting Supervisor Domain manager programs the device IDE extended capability: All RIDs and all memory is allowed, IDE is disabled for the selected stream.
3. The hosting Supervisor Domain manager implicitly initiates the IDE link setup by calling the `sbi_covh_connect_device()` COVH function, and passing the selected stream ID as an argument to it. The TSM must first establish a secured SPDMM session, as described in the previous section.

4. After it establishes a secured SPDMM session with the device, the TSM starts setting the IDE link up and programs the Root Port IDE Extended Capability with the proper RID range and the stream ID selected by the host supervisor domain manager in step 1. IDE is disabled for that stream ID. The capability is only accessible to the TSM, i.e. any writes to it with the C-bit set to 0 are dropped and reads return all 1s.
5. The TSM generates an IDE key for each sub-stream for the stream ID.
6. The TSM, for each Rx and Tx sub-stream (6 of them), programs the generated keys into the physical device:
 1. Generates and sends an **IDE_KM KEY_PROG** message to the DSM. The message is encapsulated in a vendor-defined SPDMM request.
 2. Receives an **IDE_KM_KEY KP_ACK** from the DSM.
7. The TSM, for each Rx and Tx sub-stream (6 of them), programs the generated keys into the physical device's PCIe root port, by calling the **sbi_covt_program_rp_ide_key()** COVT functions.
8. The TSM, for each Rx sub-stream (3 of them), triggers IDE in the physical device:
 1. Generates and sends an **IDE_KM SET_GO(Rx)** message to the DSM. The message is encapsulated into a vendor-defined SPDMM request.
 2. Receives an **IDE_KM_KEY K_GOSTOP_ACK** from the DSM.
9. The TSM, for each Rx sub-stream (3 of them), triggers IDE in the physical device's PCIe root port by calling the **sbi_covt_set_go_rp_ide_key()** COVT function.
10. The TSM, for each Tx sub-stream (3 of them), triggers IDE in the physical device:
 1. Generates and sends an **IDE_KM SET_GO(Tx)** message to the DSM. The message is encapsulated into a vendor-defined SPDMM request.
 2. Receives an **IDE_KM_KEY K_GOSTOP_ACK** from the DSM.
11. The TSM, for each Tx sub-stream (3 of them), triggers IDE in the physical device's PCIe root port by calling the **sbi_covt_set_go_rp_ide_key()** COVT function.

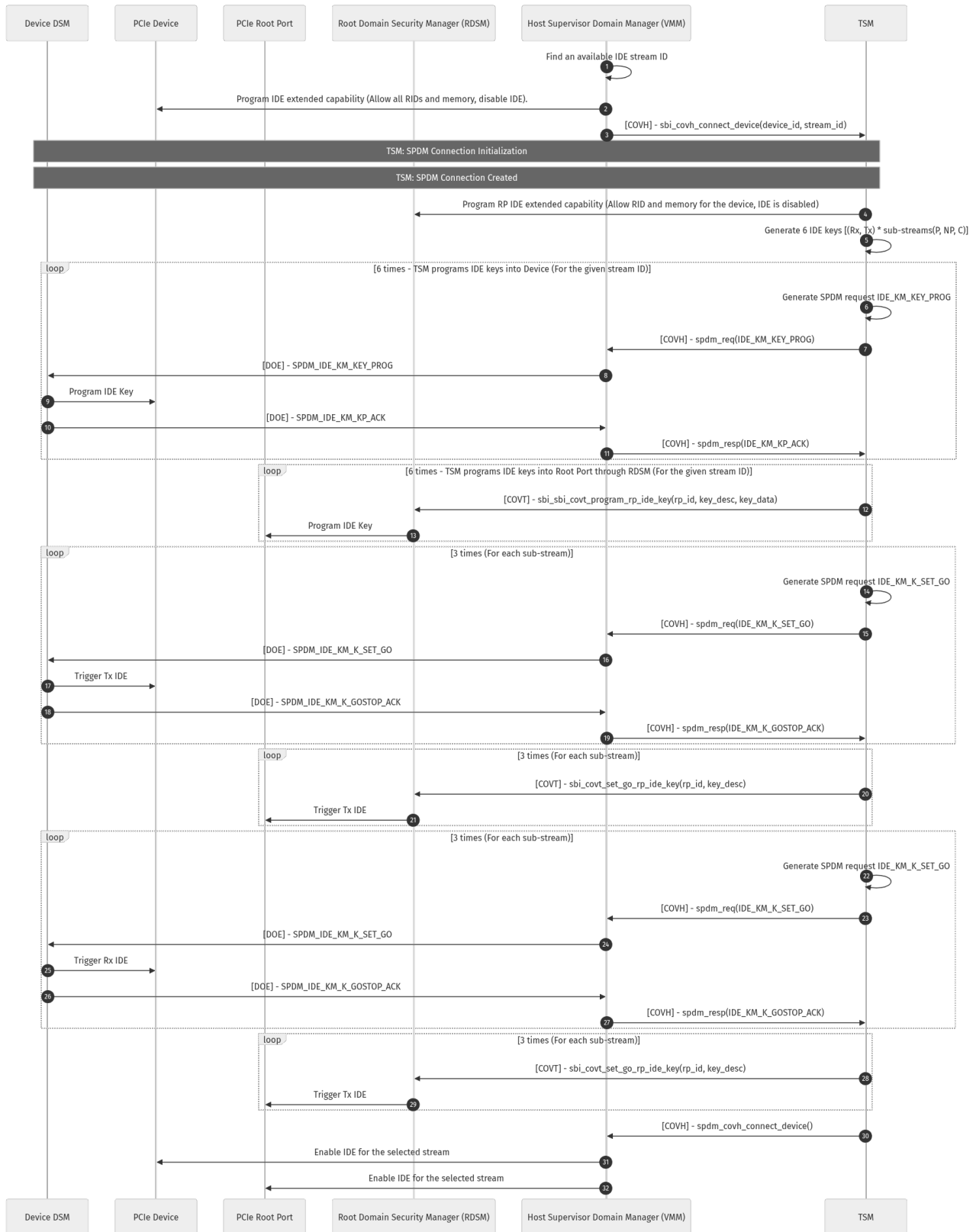


Figure 7. Device Connection - IDE Link Setup

7.5. Device Disconnection

The hosting Supervisor Domain manager may decide to reclaim all resources related to a connected device. All TDIs tracking states, TDISP resources, SPDM session context or allocated IDE stream can be reclaimed by disconnecting the TSM from the device. This is achieved through a `sbi_covh_disconnect_device()` COVH ABI call to the connected TSM.

TSMs servicing the `sbi_covh_disconnect_device()` request must return an error if some TDIs belonging to the related device are still bound to one or more TVMs managed by the TSM. The hosting Supervisor Domain manager must first unbind all TDIs from their respective TVMs before attempting to disconnect the device from the TSM.

To bind again a TDI from the disconnected device to a TVM, the host supervisor domain manager must first [establish a new connection](#) between the device's DSM and the related TSM.

7.5.1. Disconnection Flow

Upon receiving a device disconnection request from the hosting Supervisor Domain manager, the TSM must end the previously established SPDM session with the device DSM. Before doing so, the TSM must use the SPDM session to disable the IDE stream between the PCIe root port and the device, and clear all IDE key material:

1. The hosting Supervisor Domain manager [unbinds all TDIs](#) assigned to any TVM managed by the TSM that owns the connection with the physical device.
2. The hosting Supervisor Domain manager disables IDE from the device's PCIe selective IDE stream extended capability control register.
3. The hosting Supervisor Domain manager calls the `sbi_covh_disconnect_device()` COVH in order to initiate the device disconnection.
4. The TSM disables IDE from the Root Port's PCIe selective IDE stream extended capability control register.
5. The TSM, for each Tx sub-stream, disables the corresponding IDE key previously programmed into the physical device:
 1. Generates and sends an `IDE_KM SET_STOP(Tx)` message to the DSM. The message is encapsulated into a vendor-defined SPDM request.
 2. Receives an `IDE_KM_KEY K_GOSTOP_ACK` from the DSM.
6. The TSM, for each Tx sub-stream, disables the corresponding IDE key previously programmed into the physical device's PCIe root port by calling the `sbi_covt_set_stop_rp_ide_key()` COVT function.
7. The TSM, for each Rx sub-stream, disables the corresponding IDE key previously programmed into the physical device:
 1. Generates and sends an `IDE_KM SET_STOP(Rx)` message to the DSM. The message is encapsulated into a vendor-defined SPDM request.
 2. Receives an `IDE_KM_KEY K_GOSTOP_ACK` from the DSM.
8. The TSM, for each Rx sub-stream, disables the corresponding IDE key previously programmed into the physical device's PCIe root port by calling the `sbi_covt_set_stop_rp_ide_key()` COVT function.
9. The TSM clears all stored IDE keys and state.
10. The TSM terminates the SPDM session with the physical device:
 1. Generates and sends an SPDM `END_SESSION` message to the DSM.
 2. Receives an SPDM `END_SESSION_ACK` message from the DSM.
11. The TSM clears all SPDM session secrets and data.

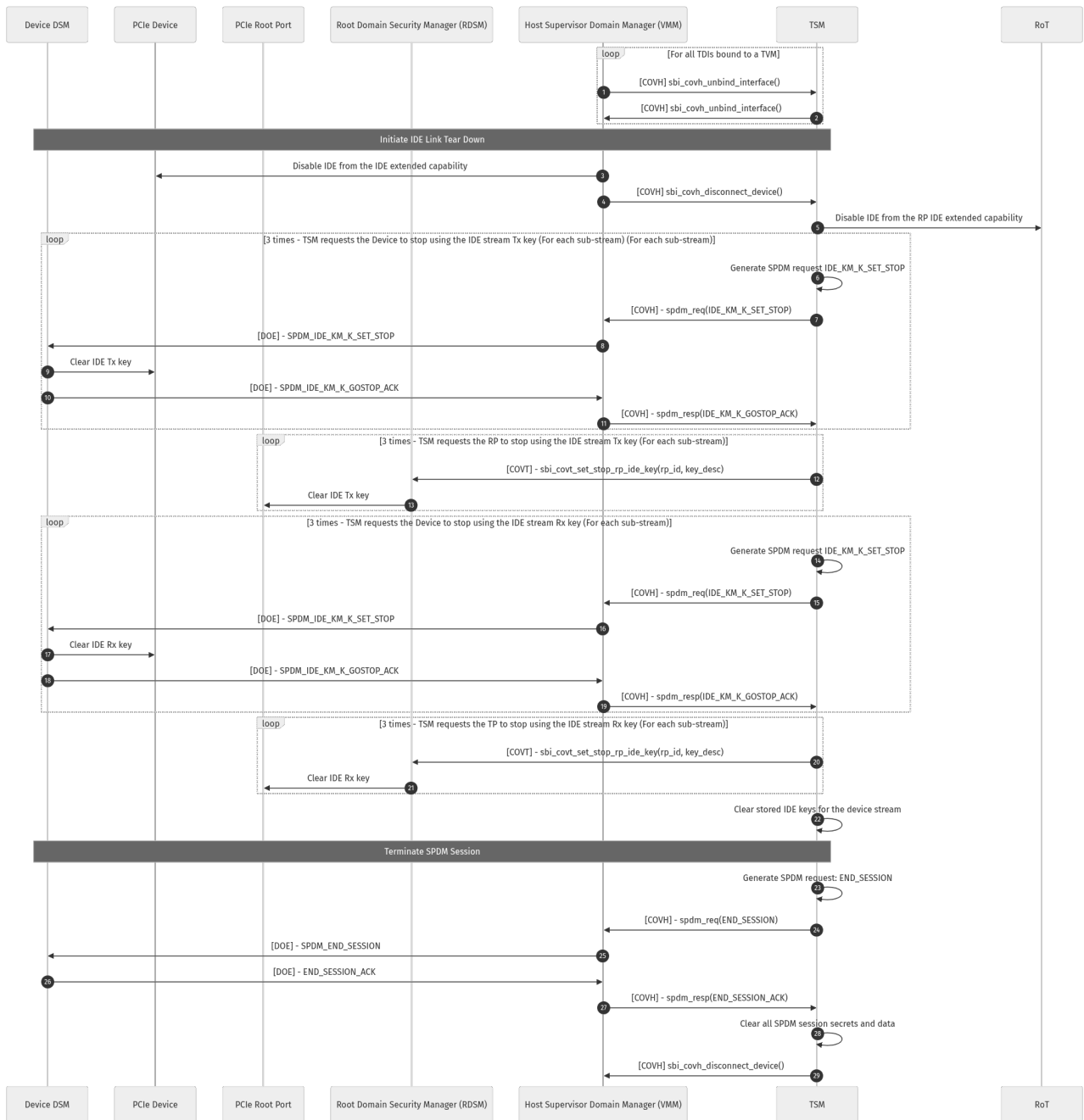


Figure 8. Device Disconnection

7.6. Interface Binding

Once both the SPDM session and the IDE link are secured and established, the hosting Supervisor Domain manager may bind a TDI and a TVM together, through the **COVH** interface. This is a four steps process:

1. The hosting Supervisor Domain manager initiates the interface binding flow by having the TSM move the TDI into the TDISP **CONFIG_LOCKED** state. This is achieved through the **sbi_covh_bind_interface()** COVH ABI.
2. The TVM **verifies and accepts the locked TDI** into its TCB.
3. The TVM asks the TSM to move the TDI to the TDISP **RUN** state, by calling the **sbi_covg_start_interface()** COVG ABI.
4. The TVM verifies that the TDI is in the TDISP **RUN** state and starts using it. This verification is provided by the **sbi_covg_get_interface_state()** COVG ABI.

The next two sections respectively give a detailed description of the overall process for binding an interface to a TVM, and one of its most critical steps: the TVM decision of accepting or rejecting the TDI into its TCB.

7.6.1. Binding Flow

Binding an interface and a TVM together goes through the following steps:

1. For the binding process to succeed, the hosting Supervisor Domain manager must first pass each of the bound TDI MMIO ranges to the TVM. It does so by calling the `sbi_covh_add_tvm_interface_region()` COVH ABI. The TSM services those requests by creating uncacheable G-stage mappings for each range, so that the TVM outbound transaction will not trigger MMIO page faults. The TSM must not enable the above mappings until the TVM accepts the TDI in its TCB, by moving it to the TDISP RUN state.
2. The hosting Supervisor Domain manager calls the `sbi_covh_bind_interface()` COVH function to start binding the TDI and the TVM together. The TSM rejects that request if a secured SPDm session is not established with the DSM or if a stream IDE link is not set up.
3. The TSM discovers and queries the TDI's TDISP version and capabilities by respectively generating and sending the TDISP `GET_TDISP_VERSION` and `GET_TDISP_CAPABILITIES` messages to the DSM. The respective `TDISP_VERSION` and `TDISP_CAPABILITIES` responses let the TSM select a common TDISP version and set of capabilities to be used in all future TDISP communication with the DSM.
4. The TSM locks the bound TDI by generating and sending a TDISP `LOCK_INTERFACE_REQUEST` message to the DSM, which is encapsulated into a vendor-defined SPDm request.
5. The TSM receives the TDISP `LOCK_INTERFACE_RESPONSE` message, which contains a device-generated nonce. The TSM stores the nonce. The TDI is now in the TDISP `CONFIG_LOCKED` state.
6. The TSM programs the DMA mappings into the trusted IOMMU space, in order for the TDI inbound transactions to map into the TVM address space. The IOMMU mappings are programmed but not validated. They will be validated only once the TDI moves to the TDISP RUN state, i.e. when it accepts the TDI into its TCB through the `sbi_covg_start_interface()` COVG ABI.
7. Through regular discovery mechanisms (ACPI, PCI bus scanning), the TVM detects the assigned TDI. It is important to note that the TDI configuration space, including its BARs, is emulated by the hosting Supervisor Domain manager. The TDI MMIO ranges are mapped into the TVM address space by the hosting Supervisor Domain manager, through the `sbi_covh_add_tvm_interface_region()` COVH ABI.
8. Before using the TDI, the TVM must [accept it into its TCB](#). Moreover, the TVM must not use the TDI until it can verify from the TSM that it's been put in the TDISP RUN state (step 12 below). To notify the TSM about its decision to accept or reject the bound TDI, the TVM respectively call into the `sbi_covg_start_interface()` or `sbi_covg_stop_interface()` COVG ABI.
9. Upon acceptance of the TDI by the TVM, the TSM generates and sends a TDISP `START_INTERFACE_REQUEST` message that includes the `LOCK_INTERFACE_RESPONSE` nonce received on step 5. The message is encapsulated into a vendor-defined SPDm request.
10. The TSM receives the TDISP `START_INTERFACE_RESPONSE`. The TDI is now in the TDISP RUN state.
11. The TSM enables the IOMMU and the TVM G-stage mappings configured on steps 6 and 1, for respectively enabling the DMA and MMIO operations with the bound TDI.
12. The TVM verifies that the TDI is in the TDISP RUN state by calling the `sbi_covg_get_interface_state()` COVG ABI.
13. The TVM can start using the device.

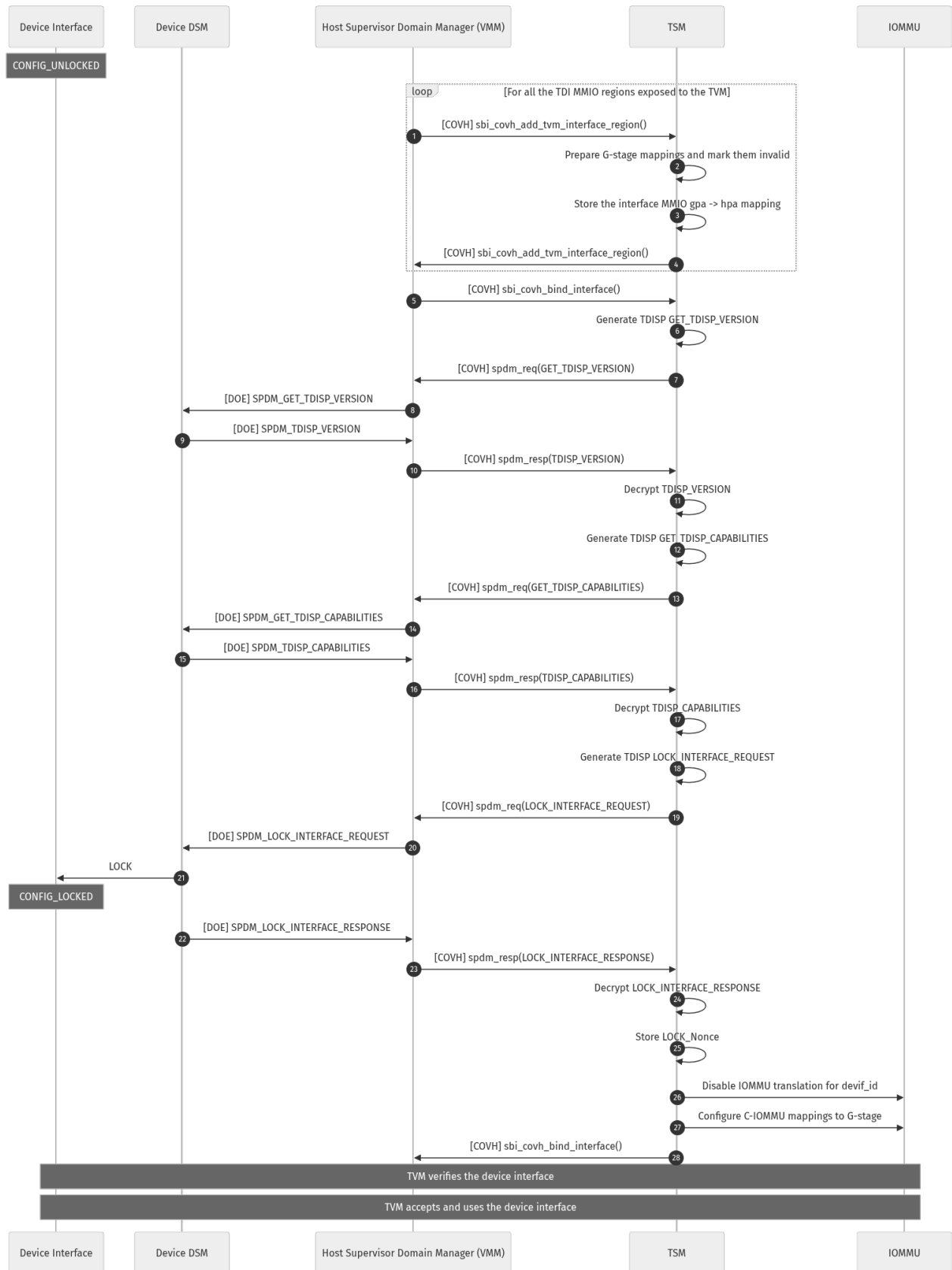


Figure 9. Device Interface Binding - Bind Interface

7.6.2. TDI Verification and Acceptation

It is the TVM responsibility to accept or reject the assigned TDI into its TCB, and to explicitly notify both the TSM and the hosting Supervisor Domain manager about its decision. The TVM should verify the following security attributes before being able to decide whether or not it can safely accept a TDI into its TCB:

1. **SPDM session establishment:** A secured SPDM session must be established between the TDI's DSM and the TSM. TVM verifies that attribute from the TSM, through the `sbi_covg_get_device_link()` COVG ABI.
2. **IDE link:** The PCIe physical link between the Root Port and the physical device must be confidentiality and integrity protected through IDE. As for the SPDM session, the TVM calls into the `sbi_covg_get_device_link()` COVG ABI to verify that attribute from the TSM.
3. **TDISP and SPDM configuration:** The TVM must verify that the TDI TDISP configuration and the SPDM session attributes comply with its security policy. For example, the TVM could check for the allowed device firmware update policy by combining the TDI TDISP report `NO_FW_UPDATE` setting with the SPDM session measurement freshness capabilities (`MEAS_FRESH_CAP`). It is then the TVM choice to accept or reject a TDI depending on the inferred physical device firmware update policy. The TDI interface report and the SPDM session attributes are provided by respectively the `sbi_covg_get_interface_report()` and the `sbi_covg_get_device_spdm_attrs()` COVG ABI.
4. **TDI state:** Before accepting a TDI into its TCB, a TVM must verify that its configuration is immutable, and in particular that the host can not modify it without having all in-flight transactions being discarded. TEE-IO capable physical devices follow the TDISP specification and can guarantee that immutability state once the TDI has been moved to the TDISP `CONFIG_LOCKED` state. The transition from TDISP `CONFIG_UNLOCKED` to `CONFIG_LOCKED` is triggered by the hosting Supervisor Domain manager through the COVH ABI. As such, the TVM can query the TSM for the TDI state through the `sbi_covg_get_interface_state()` COVG ABI. A TVM must not accept a TDI if it's in any other TDISP state than `CONFIG_LOCKED`.
5. **Device trustworthiness:** Verifying that the TDI is in an immutable state across a secured SPDM and physical link is mandatory but not sufficient. The TVM must also attest to the physical device trustworthiness in order to decide if it can accept one of its TDIs into its TCB. A TVM can trust a PCIe device by first authenticating it. Once authenticated, the TVM challenges the device and then verifies its measurements:
 1. First the TVM must first verify the authenticity of the device by getting its certificate chain from the TSM, through the `sbi_covg_get_device_certificate()` COVG ABI. The TVM should then verify the chain against a provisioned and measured trust anchor list.
 2. Once the device certificate authenticity is verified, the TVM must then challenge it by having it sign a piece of data, making sure that the device actually owns the private key bound to its certificate. This is achieved by getting the TDI measurements from the TSM through the `sbi_covg_get_device_measurement()` COVG ABI. This set of device-signed measurements, also known as the device attestation evidence, must be verified against the TDI certificate acquired in the previous step.
 3. Finally, the TVM should attest to the device configuration trustworthiness (code, SVN, state, etc) by verifying the previously fetched device attestation evidence. This is typically done through a remote or local attestation procedure.
6. **TDI IO ranges:** The TVM will likely interact with and program the TDI through a set of memory mapped IO ranges (e.g. a PCI BAR defined memory range). However, when discovering the TDI in its address space, the TVM only sees guest physical addresses (GPA) for those ranges, as exposed by the hosting Supervisor Domain manager PCI emulation. When communicating with the TDI, the TVM will use those GPAs and must rely on their corresponding translations to host physical addresses (HPA) to be properly set. In particular, it must rely on the fact that the TDI MMIO ranges GPAs do not map to non-confidential memory that could be otherwise accessed by a host domain component. To verify that security attribute, the TVM must retrieve the TDISP report for the TDI, through the `sbi_covg_get_interface_report()`. The TDISP report, among other things, contains the list of MMIO ranges for the TDI sorted by BAR indexes. First, the TVM must verify that the host VMM exposed BARs have the same sizes as the TDISP reported ones. To further validate those ranges, the TVM must check from the TSM that they're correctly mapped to host physical ranges. Prior to the TVM being able to

accept a TDI, the host VMM must have requested the TSM to map all the TDI MMIO ranges to TVM GPA ranges, through `sbi_covh_add_tvm_interface_region()` COVH calls. The TVM then verifies from the TSM that a GPA exposed TDI MMIO range will be mapped to the TDISP reported range through the TSM managed G-stage page tables, by calling into the `sbi_covg_map_interface_mmio()` COVG ABI. The TVM can accept a TDI only if the TSM confirms the validity of all MMIO range mappings, in the TDISP reported order (i.e. BAR #N in the TVM address space will be mapped to the TDISP reported MMIO range #N).

Once the TVM has verified the above security attributes, it lets the TSM know that it is ready to use the TDI, by calling into the `sbi_covg_start_interface()` COVG ABI.

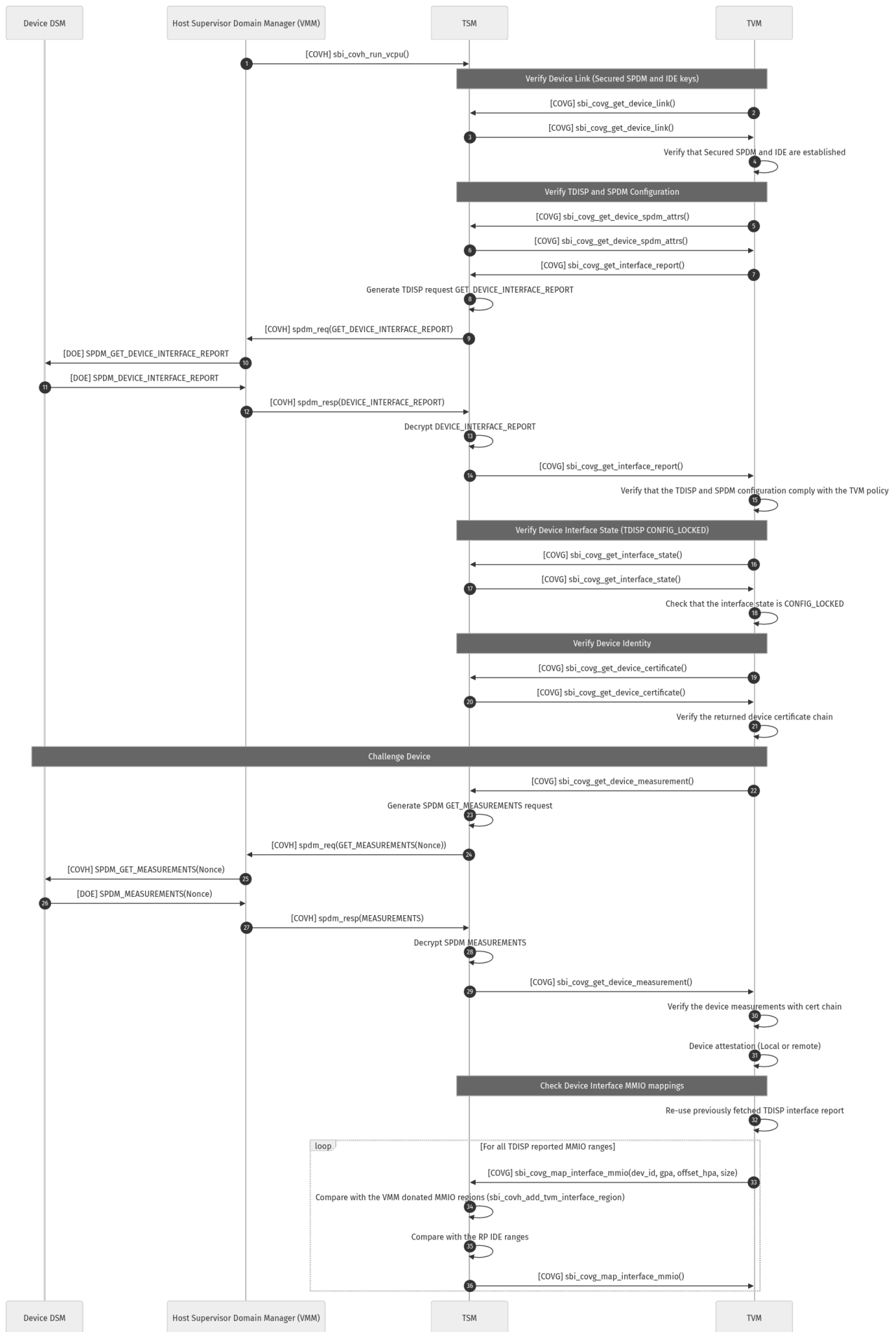


Figure 10. Device Interface Verification

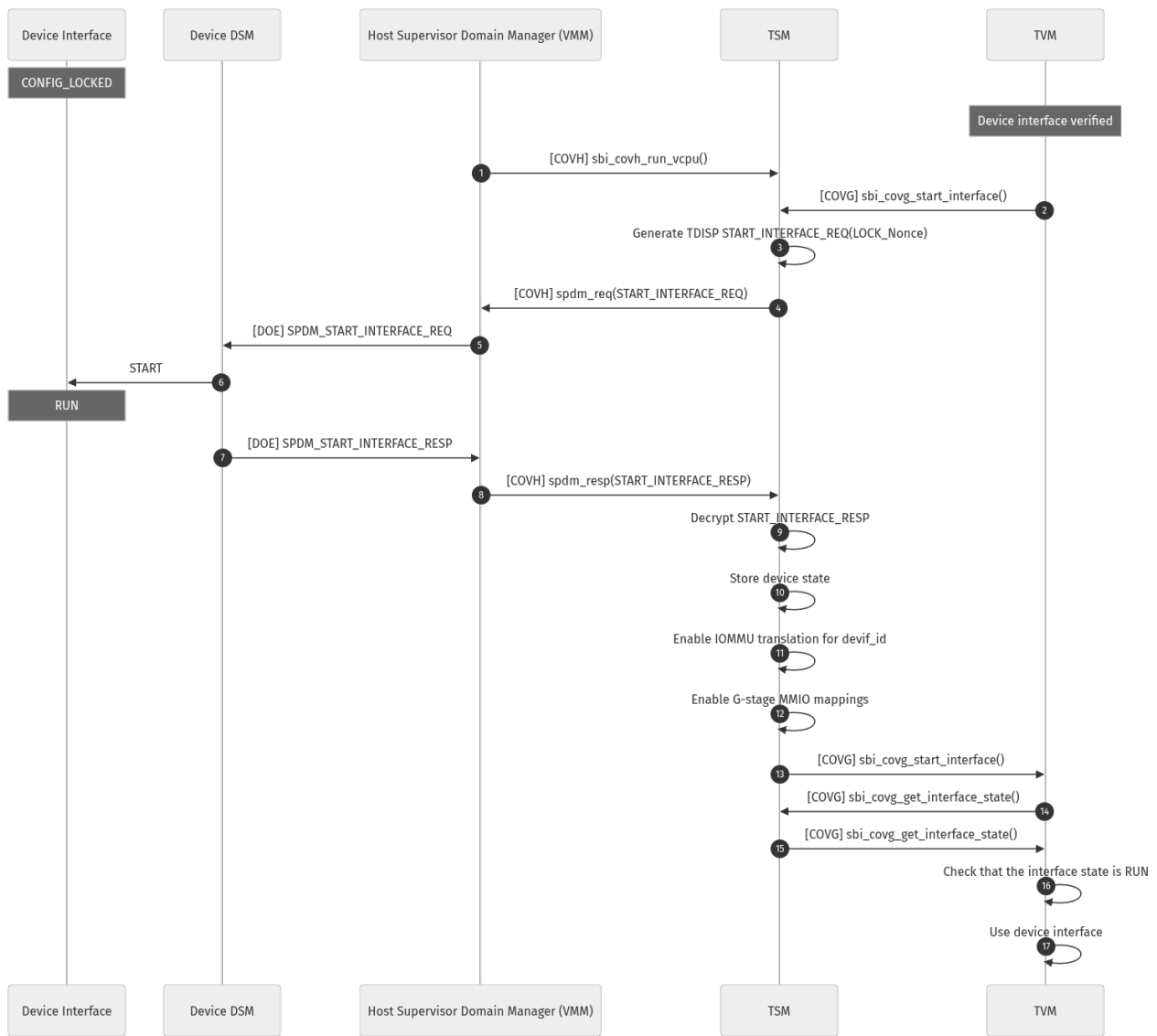


Figure 11. Device Interface Acceptation

7.7. Interface Unbinding

As the platform resources owner, only the hosting Supervisor Domain manager may decide to unbind an interface from a TVM. At any moment, it can choose to do so through the `sbi_covh_unbind_interface()` COVH ABI. The targeted TSM services the request by moving the TDI from the TDISP **CONFIG_LOCKED**, **RUN** or **ERROR** state back to the **CONFIG_UNLOCKED** one.

The hosting Supervisor Domain manager may decide to unbind a TDI from a TVM for many reasons. First of all, the TVM itself could be requesting that a TDI be unbound through the `sbi_covg_stop_interface()` COVG ABI, because e.g. it no longer needs the TDI for its operation or when shutting down. The host Supervisor Domain manager can also asynchronously choose to reclaim a TDI from a TVM, because the TVM or the TDI is misbehaving or unresponsive, or simply because it needs to re-assign a TDI to another TVM. Once unbound, a TDI can then be assigned to any TVM on any Supervisor Domain.

Unbinding a TDI from a TVM is first and foremost a resource management operation, and as such it must be designed to robustly address multiple error conditions and states that either the TDI or the TVM may have reached, while at the same time maintaining confidential data integrity and confidentiality. In order to unconditionally release all TDI related system resources, the operation does not require any interaction with neither the TDI nor the TVM to complete.

7.7.1. Unbinding Flow

Unbinding an interface from a TVM goes through the following steps:

1. The hosting Supervisor Domain manager initiates the unbinding operation through the `sbi_covh_unbind_interface()` COVH ABI. The TSM rejects that request if the targeted TDI and TVM are not bound together.
2. The TSM generates and sends the TDISP `STOP_INTERFACE_REQUEST` to the DSM.
3. The DSM and the TDI complete or abort all pending DMA requests.
4. The DSM and TDI scrub the internal state of the interface in order to remove and clear all secrets and confidential data.
5. The TSM receives the TDISP `STOP_INTERFACE_RESPONSE`. The TDI is now in the `CONFIG_UNLOCKED` state.
6. The TSM disables the C-IOMMU and the TVM G-stage mappings that were enabled upon [acceptation of the TDI by TVM](#).
7. Eventually, the hosting Supervisor Domain manager reclaims the TDI MMIO regions exposed to the TVM address space by calling the `sbi_covh_reclaim_tvm_interface_region()` COVH ABI for each and every one of them. The TSM rejects any one of those requests if the reclaimed MMIO region belongs to a bound interface, or if it does not belong to the MMIO gpa → hpa mapping the TSM initially created for the interface at [TDI binding time](#).
8. The TSM removes all C-IOMMU mappings and all cached C-IOMMU Address Translation Cache (ATC) mappings, the TVM G-stage mappings and the MMIO gpa → hpa mappings for the interface.

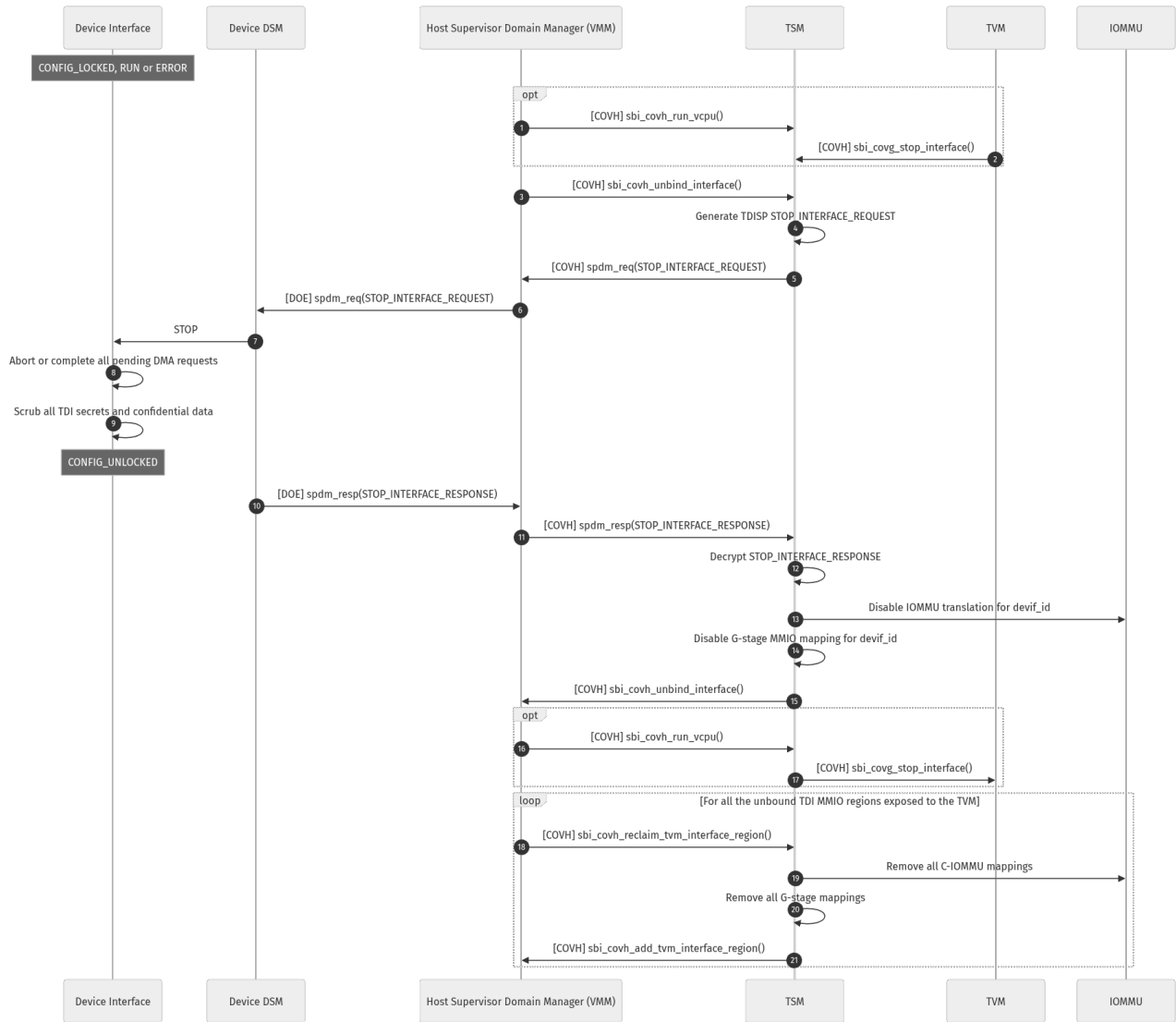


Figure 12. Device Interface Unbinding - Unbind Interface

7.8. Session Key Update

In TEE-IO architecture, when connecting to a device, a TSM establishes an SPDM session and an IDE session. Both sessions use AES-GCM-256 keys to protect the data. As such, the hosting Supervisor Domain manager needs to update the session keys before the use of keys reach to the limit.

7.8.1. SPDM Session Key Update

SPDM session key update is managed by SPDM KEY_UPDATE command. The TSM shall count the usage of SPDM session keys, also known as the 64bit **sequence number** defined in [\[SecuredSPDM\]](#).

Whenever the TSM generates and encrypts SPDM messages to support the host Supervisor Domain manager (e.g. a VMM) **COVH** requests, it shall check if the **sequence number** reaches the limit. If it does, the TSM replies to the request with the **SBI_ERR_KEY_UPDATE_REQUIRED** error in order to indicate to the VMM that it should initiate an SPDM Key Update (through the **sbi_covh_spdm_key_update()** **COVH** call).

Once the SPDM Key Update completes, the VMM shall re-send the failing **COVH** request.

7.8.2. IDE Session Key Refresh

IDE session key refresh is managed by host software via IDE_KM command. The host root port IDE engine shall count the 64-bit invocation field of the IV, check if the value reaches the limit, and then process the IDE TLP.

If the invocation field overflows, the IDE stream shall move to the insecure state and the IDE session shall terminate.

In order to keep the IDE session alive, the VMM should perform IDE Key Refresh action (through the `sbi_covh_ide_key_refresh()` COVH call) before the IDE session reaches the limit of the key usage.

Currently, there is no robust way to get the invocation field of IV for a given IDE session. One possible way is to do time-based refresh. For example, the VMM could periodically perform IDE key refresh, based on the calculation of maximum usage of IDE TLP.

By default, The maximum AES-GCM counter allowed in the algorithm is 2^{64} . [\[AEADLimit\]](#) provided a way to allow a TSM or DSM to set a smaller AES-GCM counter to resist the attack to AEAD algorithm. The TSM and DSM should support AEAD Limit Counter feature described in [\[AEADLimit\]](#).

7.9. Session Heartbeat

A device may require SPDM session heartbeat to ensure the session is kept alive on the host side. If the both TSM and DSM support SPDM heartbeat, the device may return a non-zero `HeartbeatPeriod` in `KEY_EXCHANGE_RSP` response message.

The host is required to send SPDM message within two successive `HeartbeatPeriod`'s, otherwise the device will terminate the corresponding SPDM session.

When a SPDM heartbeat is required, the VMM must initiate an SPDM heartbeat (through the `sbi_covh_spdm_heartbeat()` COVH call).

7.10. Device and Interface Lifecycle

When combined together, the flows and ABIs described in the previous sections are used to build the lifecycle of a TDISP capable device on a CoVE-IO compatible platform, as illustrated in the following figure:

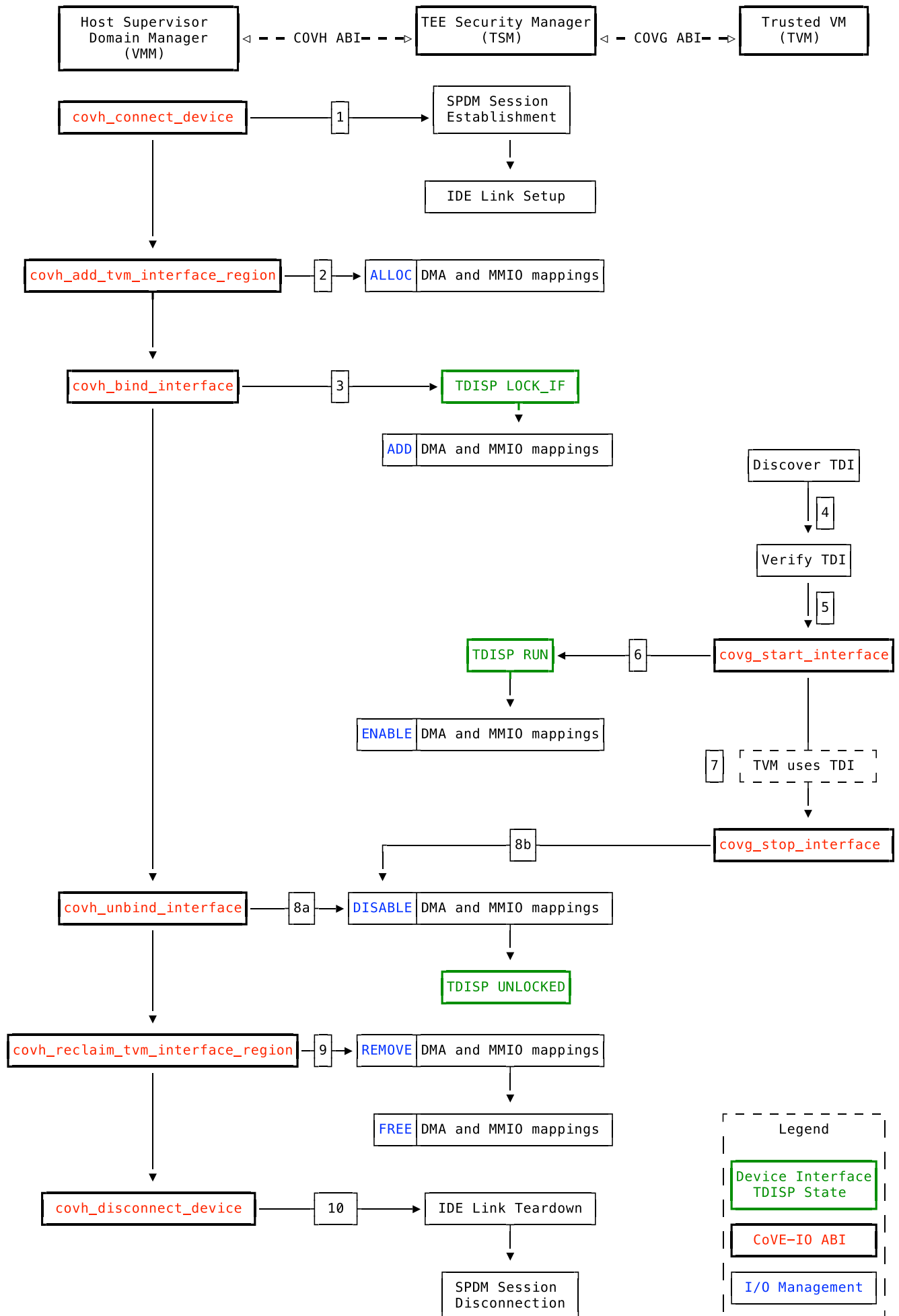


Figure 13. Device and Interface Lifecycle

The hosting Supervisor Domain manager owns the physical device, and manages its lifecycle. The TSM enforces that this lifecycle management is done without compromising any TVM confidential assets.

As the platform resources owner, the hosting Supervisor Domain manager can assign a TDI to a TVM by binding them together (step 3). At any point in time, it can reclaim that physical resource by unbinding (step 8a) it from its TVM. Graceful and explicit unbinding through the **COVH** ABI will clear all confidential data and abort any outstanding DMA requests, as described in the [Interface Unbinding](#) section. Implicit TDI unbinding, i.e. attempts from the hosting Supervisor Domain manager to reclaim and remap a TDI without explicitly notifying the corresponding TSM, represents a security threat that is in the [CoVE-IO threat model](#) scope. In that situation, both the DSM and the TSM must protect the confidentiality and integrity of the TVM assets.

Before binding a TDI and a TVM together, the hosting Supervisor Domain manager must first require the TSM to connect (step 1) to the physical device through secured SPDH. As part of servicing that request, the TSM also protects the physical link with PCIe IDE.

The hosting Supervisor Domain manager is also required to explicitly add the TDI MMIO regions to the TDI (step 2). The TSM can prepare and allocate the TVM second stage page tables and map those I/O regions into the guest physical address space. The TSM does not enable those tables until the TVM starts the interface (step 6)

Only once the TSM is securely connected to the physical device, The host Supervisor Domain manager can proceed with binding a device interface and a TVM together. Once bound to a TVM, the device interface is locked but the MMIO and DMA paths between the two are not enabled yet.

The TVM to which a TDI is bound to is the I/O gatekeeper. After detecting and verifying (steps 4 and 5) the bound interface, it may accept it into its TCB and enable all I/O paths between the two parties. The TVM may only use a bound TDI (step 7) after accepting it, by requesting the TSM to start the device interface (step 6).

Both the hosting Supervisor Domain manager and the TVM can disable I/O between the TDI and the TVM, by respectively unbinding (step 8a) or stopping the device interface (step 8b). The hosting Supervisor Domain manager may first remove the TDI MMIO regions from the TVM address space (step 9).

Finally, the hosting Supervisor Domain manager can fully reclaim the physical device by requesting the TSM to disconnect (step 10) from it.

Chapter 8. Device Attestation

There are two attestation stages involved in the CoVE-IO architecture.

- TVM attests the device
- A third party attests the TVM with the device

The IETF RATS Remote Attestation Architecture [\[RATS\]](#) can be applied to both.

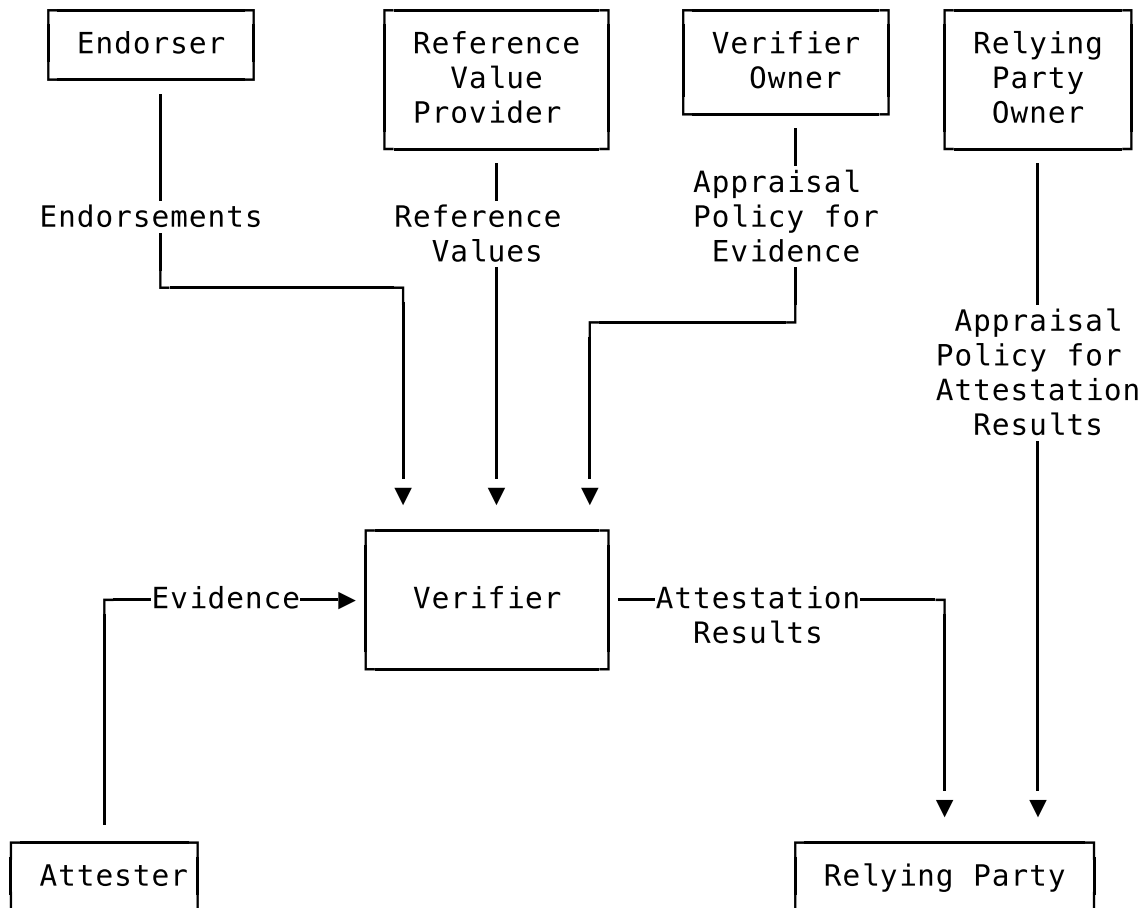


Figure 14. Remote Attestation Framework (IETF RATS)

8.1. TVM attesting the Device

The TVM must verify and trust the device before offloading the workload to the TDI. According to the [\[TDISP\]](#) architecture, the high level steps for that stage are as follows:

1. The TSM collects the device certificate and measurement from the DSM as the evidence, via SPDM protocol.
2. The TSM presents the evidence to the TVM.
3. A TVM verifier appraises the evidence and produce the attestation result to the relying party in the TVM, such as device driver.
4. The local relying part running in the TVM accepts (or rejects) the device into (from) its TCB.

The above process follows the background-check model, as defined by [\[RATS\]](#). The [\[RATS\]](#) passport model is not supported by the CoVE-IO attestation architecture, because it adds complexity to the TSM as the TSM would need to perform the verification and pass the attestation result to TVM. Since the TSM is in the

TVM's TCB, its attack surface should be kept minimal.

In step 3, the verifier needs to perform the verification for the device. There are two possible use cases:

8.1.1. Local Verification

With local verification, the endorsement and reference values are provisioned into the TVM.

Static provisioning means the data is built into the TVM image at build time, while dynamic provisioning means the data is provisioned into the TVM at runtime.

The endorsement may include the device root CA certificate and its associated certificate revocation list (CRL). The reference value includes the reference integrity manifest (RIM), also known as golden measurement.

When running locally, the verifier in the TVM validates the device certificate according to the device root CA certificate and CRL, and then compares the device measurement with the measurement from its locally provisioned RIM.

The TVM may have a local policy update mechanism to update the endorsement and reference value. How to trigger the update is out of scope of this specification. It could be a pull model, such as TVM querying for updates periodically. Or it could be a push mode, such as orchestrator posting the update when it notices the device vendor update. To maintain the integrity of the policy update, the update package should have protection mechanism, such as digital signature.

8.1.2. Remote Verification

When the device is verified remotely, the endorsement and reference values are not provisioned into the TVM, but are accessible from an externally available service, also known as a relying party for the device.

The verifier itself can either run locally or be hosted on a remote service as well, typically available through an HTTPS-based protocol. It always collects the up-to-date endorsement and RIM from the device relying party.

Table 12. Device Attestation Comparison

Use Case	Local Verification	Remote Verification
Endorsement (Root CA Cert, CRL)	Provisioned in TVM (Static or Dynamic)	Remote Service
Reference Value (RIM)	Provisioned in TVM (Static or Dynamic)	Remote Service
Policy Update (Endorsement, Reference)	No update or scheduled update (Pull or Push)	Always up-to-date
Verifier Location	Inside of TVM	Inside of TVM or Remote Service

8.1.3. Device Runtime Update and Reattestation

A TEE-IO device may support runtime update. Currently, there are two policy settings controlling device runtime update.

SPDM Session TerminationPolicy

This SPDM session attribute controls if runtime update will cause SPDM session termination or device may keep the session.

Note The session termination means the IDE stream will change to **insecure** state and TDI in **ERROR** state.

The SPDM **TerminationPolicy** is set in SPDM **KEY_EXCHANGE** message by TSM. The TSM should return such policy information to the TVM via **sbi_covg_get_device_spdm_attrs()** COVG ABI.

TDISP NO_FW_UPDATE

This TDISP interface attribute controls if runtime update is allowed after TDI in either **CONFIG_LOCKED** or **RUN** state.

Note Runtime update is always allowed if TDI is in **CONFIG_UNLOCKED** or **ERROR**.

The TDISP **NO_FW_UPDATE** is set in TDISP **LOCK_INTERFACE_REPORT** message by TSM, and is returned in **BIT0** of **INTERFACE_INFO** in **TDI Report Structure** by the device. The TVM can get the full **TDI Report Structure** via the **sbi_covg_get_interface_report()** COVG ABI.

The TVM should consult both setting to decide if it can accept the device runtime update.

If the TVM allows the device runtime update after lock (**NO_FW_UPDATE=0**) and update without session termination (**TerminationPolicy=1**), then the TVM may want to do attestation again to verify the device.

There are 2 possible models.

Post-update reattestation

After the device performs the update, the TVM will collect both the certificate and the measurement and do verification again to determine if it still wants to accept the device. If the TVM decides to no longer accept the device any more, the TVM will use **sbi_covg_stop_interface()** ABI to request the VMM to use **sbi_covh_unbind_interface()** to unbind the TDI, then the TVM will verify with TSM if the TDI is really unbound by the VMM.

How the device performs the update is out of scope of this specification. It could be initiated by current TVM, other TVMs, other legacy VMs, VMM, or even out of band (OOB) mechanism, such as Baseboard Management Controller (BMC).

When the TVM performs the reattestation is also out of scope of this specification. It could be a pull mode with policy inside of TVM, such as reattestation every week. Or it could be an interrupt mode such as request triggered by the orchestrator.

Note In this mode, the device can be updated before the TVM performs the reattestation, or just after TVM performs reattestation. If this is a concern, the TVM should choose no-update or pre-update attestation.

Pre-update attestation

With post-update reattestation, there is a gap between update time and attestation time. Confidential data access from the TDI may happen before the updated device is re-attested and re-accepted by the TVM.

SPDM 1.3 added an optional event mechanism and **MeasurementPreUpdate** event notification. If the device, TSM, and TVM all support SPDM 1.3, the TVM may get the **MeasurementPreUpdate** event notification and verify the measurement before the update takes effect. If the TVM cannot accept the new measurement, then the TVM can unbind the TDI before runtime update.

The difference between this pre-update attestation and post-update reattestation is that the update takes effect when the verification happens. Pre-update attestation gives a chance to unbind TDI before runtime update, while post-update reattestation can only unbind TDI after runtime update. That can remove the gap between update time and attestation time.

Note In this mode, the device will be updated, regardless of the TSM or the TVM accepting the firmware update or not. If TSM cannot accept the new firmware, the TSM shall terminate the SPDM session with SPDM message **END_SESSION**, but not acknowledge the SPDM event with SPDM message **EVENT_ACK**. If TVM cannot accept the new firmware, the TVM shall unbind the TDI.

8.2. Third party attesting the TVM with the device

The third party must follow the CoVE defined attestation mechanism to perform local or remote attestation and verify the TVM.

The CoVE-IO architecture adds the additional device related information to the TVM attestation evidence, in order to allow the third party to verify the attached device TDI.

Depending on the verification process being local or remote, the TVM needs to add different pieces of device-related information to the TVM report:

8.2.1. Local Verification

The TVM report should include the device verifier code, the device policy, including the provisioned device root CA certificate, CRL, and RIM.

If the TVM supports device policy update, then the secure signed update mechanism should be implemented. In that case, the signer of the update data shall be included in the TVM report as the trust anchor.

The device measurement and certificate are not required in the TVM report, however the TVM should provide a mechanism to return the device measurement and certificate to the verifier for further verification.

8.2.2. Remote Verification

The TVM report should include measurements of either the device verifier or the proxy connecting to the verifier service. It should also include the remote service URL and the public certificate as the trust anchor.

The detailed, up-to-date endorsement or reference value is **not** required to be part of the TVM report.

Although the device measurement and certificate are not required to be included in the TVM report, the TVM should provide a mechanism to return the device measurement and certificate for the verifier to perform further verification.

To support remote verification, the device measurement data shall be the signed [SPDM] measurement transcript, including **VCA** and all {**GET_MEASUREMENTS**, **MEASUREMENTS**} pairs that are exchanged between the SPDM measurement requester and the responder. Only the last **MEASUREMENTS** shall include the digital signature of the measurement transcript.

Providing the signed [SPDM] measurement transcript has multiple benefits:

- Measurement record integrity protection. The provided [SPDM] measurement transcript digital signature protects the measurement record integrity against:
 - Transport attacks between the host and the remote verifier, that should otherwise be protected through TLS.
 - Internal device attacks and vulnerabilities. The Device Security Manager (DSM) may be composed of several pieces of firmware, and every one of them can potentially forge the measurements before returning it to the requester. With a digitally signed measurement transcript, they can not be modified after the DSM RoT signs them, effectively removing all other DSM components out of the overall TCB.
- Additional data for attestation.
 - [SPDM] **MEASUREMENTS** response opaque data field. With a signed transcript, device-specific opaque data is included into the **MEASUREMENTS** response. This piece of data may be required by the device verifier.
 - [SPDM] connection parameter in **VCA**. The verifier can check the negotiated SPDM version, capabilities and algorithms.

Table 13. TVM Attestation Comparison

Use Case	Local Verification	Remote Verification
Device Verifier Code in TVM report	TVM Verifier	TVM Verifier or Stub Function to Verifier Service
Device Policy Data in TVM report	Endorsement and Reference Value	Remote Service URL and Public Cert (trust anchor)
Device Policy Update in TVM report	Signer of Update (as trust anchor)	N/A
Device identity NOT in TVM report	Device Measurement and Certificate	Device Measurement and Certificate

Chapter 9. Confidential VM Extension (CoVE) IO ABI

9.1. CoVE IO Function IDs Namespace

As a [\[CoVE\]](#) SBI extension, CoVE-IO has a reserved CoVE Function ID (FID) namespace. For all three CoVE EID (**COVH**, **COVI** and **COVG**), the CoVE-IO FIDs must be in the **[1024:1087]** range.

9.2. CoVE IO Host Extension (EID #0x434F5648 "COVH")

9.2.1. Physical Device Management

Function: CoVE Host Connect Device (FID #1024)

```
struct sbiret sbi_covh_connect_device(unsigned long device_id,
                                     unsigned long ide_stream_desc,
                                     unsigned long connection_data_addr_out,
                                     unsigned long connection_data_size);
```

Establishes a secure SPDH session between the TSM and a device identified by **device_id**, and configure the PCIe IDE link between that device and its upstream Root Port. The TSM must configure the IDE link according to the IDE stream description in **ide_stream_desc**.

The **ide_stream_desc** IDE stream descriptor must be structured as follows:

Table 14. CoVE IDE Stream Descriptor

Byte Offset	Field	Size (bytes)	Description
00h	Stream_ID	1	IDE selective stream ID
01h	Num_Addr_Assoc	1	Number of IDE Address Association Register blocks. Num_Addr_Assoc must not be greater than the corresponding value programmed in the device_id PCIe Selective IDE Stream Capability Register.
02h	Reserved	2	Must be 0
04h	RID_Assoc_1	4	RID Association Register 1 (See [PCIe])
08h	RID_Assoc_2	4	RID Association Register 2 (See [PCIe])
0Ch	ADDR_O_Assoc_1	4	Address Association Register 1 for block 0 (See [PCIe])
10h	ADDR_O_Assoc_2	4	Address Association Register 2 for block 0 (See [PCIe])

Byte Offset	Field	Size (bytes)	Description
14h	ADDR_O_Assoc_3	4	Address Association Register 3 for block 0 (See [PCIE])
...	...	4	...
$OCh + ((N - 1) * OCh)$	ADDR_N-1_Assoc_1	4	Address Association Register 1 for block Num_Addr_Assoc - 1 (See [PCIE])
$10h + ((N - 1) * OCh)$	ADDR_N-1_Assoc_2	4	Address Association Register 2 for block Num_Addr_Assoc - 1 (See [PCIE])
$14h + ((N - 1) * OCh)$	ADDR_N-1_Assoc_3	4	Address Association Register 3 for block Num_Addr_Assoc - 1 (See [PCIE])

The TSM returns the device connection transcript data to the host Supervisor Domain manager at **connection_data_addr_out**, structured as follows:

Table 15. CoVE Device Connection Transcript

Byte Offset	Field	Size (bytes)	Description
0	COVEIOVersion	1	CoVE-IO version
1	ConnectionTranscriptLength	4	Size of the ConnectionTranscript field that follows, in bytes
5	ConnectionTranscript	ConnectionTranscriptLength	Device connection transcript

connection_data_addr_out must be 4KB-aligned and points to a memory region that is accessible to both the host Supervisor Domain manager and the TSM.

The possible error codes returned in **sbiret.error** are shown below.

Table 16. CoVE Host Connect Device

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_INVALID_PARAMS	The device_id index is invalid.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE Host Disconnect Device (FID #1025)

```
struct sbiret sbi_covh_disconnect_device(unsigned long device_id);
```

Disconnects the TSM from the device identified by **device_id**.

The TSM must first stop all TVM-bound interfaces belonging to **device_id**, clear the IDE link and keys for **device_id** and then terminate the secure SPDH session established with **device_id**.

The possible error codes returned in **sbiret.error** are shown below.

Table 17. CoVE Host Disconnect Device

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_INVALID_PARAMS	The device_id index is invalid.
SBI_ERR_FAILED	The operation failed for unknown reasons.

9.2.2. TVM Memory Management

Function: CoVE Host Add TVM Interface Region (FID #1026)

```
struct sbiret sbi_covh_add_tvm_interface_region(unsigned long tvn_id,
                                              unsigned long device_if_id,
                                              unsigned long base_addr,
                                              unsigned long region_len);
```

Adds a device interface MMIO region into a TVM address space.

The host Supervisor Domain manager must call this function for all **device_if_id** MMIO regions before binding it to **tvn_id**. This function returns an error if **device_if_id** is already bound to **tvn_id**.

The TSM creates uncacheable G-stage mappings for the added region, so that the TVM outbound transaction will not trigger MMIO page faults. The TSM must not enable the created mappings until the TVM accepts the TDI in its TCB.

The possible error codes returned in **sbiret.error** are shown below.

base_addr value must be 4KB-aligned.

region_len value must be 4KB-aligned.

Table 18. CoVE Host Add TVM Interface Region

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE Host Reclaim TVM Interface Region (FID #1027)

```
struct sbiret sbi_covh_reclaim_tvm_interface_region(unsigned long tvn_id,
                                                    unsigned long device_if_id,
                                                    unsigned long base_addr,
                                                    unsigned long region_len);
```

Reclaims a device interface MMIO region previously added to a TVM address space.

The host Supervisor Domain manager may call this function when the **tvn_id** TVM stops the

device_if_id device interface, in order for the TSM to unmap the device interface MMIO ranges from the TVM address space.

If **device_if_id** is still bound to **tvm_id** when this call is made, the TSM unbinds the device interface from the TVM first.

The possible error codes returned in **sbiret.error** are shown below.

base_addr value must be 4KB-aligned.

region_len value must be 4KB-aligned.

Table 19. CoVE Host Reclaim TVM Interface Region

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_FAILED	The operation failed for unknown reasons.

9.2.3. Device Interface Management

Function: CoVE Host Bind Interface (FID #1028)

```
struct sbiret sbi_covh_bind_interface()(unsigned long tvm_id,
                                       unsigned long device_if_id);
```

Binds a TVM and a device interface together.

The TSM returns an error if a secured SPDm session is not established with the DSM or if a stream IDE link is not set up.

After this calls completes successfully, the **device_id_interface** is in the TDISP **CONFIG_LOCKED** state.

The possible error codes returned in **sbiret.error** are shown below.

Table 20. CoVE Host Bind Interface

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE Host Unbind Interface (FID #1030)

```
struct sbiret sbi_covh_unbind_interface()(unsigned long tvm_id,
                                          unsigned long device_if_id);
```

Unbinds a device interface from a TVM.

After this calls completes successfully, the **device_id_interface** is moved back to the TDISP **CONFIG_UNLOCKED** state, from one of the **CONFIG_LOCKED**, **CONFIG_UNLOCKED** or **CONFIG_ERROR** states.

The possible error codes returned in **sbiret.error** are shown below.

Table 21. CoVE Host Unbind Interface

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE Host SPDMM Key Update (FID #1031)

```
struct sbiret sbi_covh_spdm_key_update(unsigned long device_id);
```

Perform an SPDMM key update for a SPDMM connection between the TSM and a connected device.

The possible error codes returned in **sbiret.error** are shown below.

Table 22. CoVE Host SPDMM Key Update

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE Host IDE Key Refresh (FID #1032)

```
struct sbiret sbi_covh_ide_key_refresh(unsigned long device_id,
                                       unsigned long stream_id);
```

Perform an IDE key refresh for a connected device IDE stream.

The possible error codes returned in **sbiret.error** are shown below.

Table 23. CoVE Host IDE Key Refresh

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE Host SPDMM Heartbeat (FID #1033)

```
struct sbiret sbi_covh_spdm_heartbeat(unsigned long device_id);
```

Perform an SPDMM session heartbeat for a SPDMM connection between the TSM and a connected device.

The possible error codes returned in **sbiret.error** are shown below.

Table 24. CoVE Host SPDMM Heartbeat

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE Host Device Connection Transcript (FID #1034)

```
struct sbiret sbi_covh_get_device_connection_transcript(unsigned long
device_id,
                                                    unsigned long
transcript_data_addr_out,
                                                    unsigned long
transcript_data_size);
```

Pass the device connection transcript data back to the TSM for a given connected device. The data is formatted according to the [Device Connection Transcript](#function-cove-host-connect-device-fid-1026) structure.

transcript_data_addr_out must be 4KB-aligned and points to a memory region that is accessible to both the host Supervisor Domain manager and the TSM.

The possible error codes returned in **sbiret.error** are shown below.

Table 25. CoVE Device Connection Transcript

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_INVALID_PARAMS	Invalid parameter value provided for device_id .
SBI_ERR_FAILED	The operation failed for unknown reasons.

9.3. CoVE IO Guest Extension (EID #0x434F5647 "COVG")

9.3.1. Physical Device Query

Function: CoVE Guest Get Device Link (FID #1024)

```
struct sbiret sbi_covg_get_device_link(unsigned long device_if_id);
```

Gets the status of the link between the physical device hosting **device_if_id** and the TVM. This covers both the SPDM and IDE links.

Returns the a link status bitmap through **sbiret.value**.

```
// A secure SPDM Session is established.
#define SECURED_SPDM (1 << 0)

// A PCIe IDE link is established.
```

```
#define PCIE_IDE (1 << 1)
```

The possible error codes returned in `sbiret.error` are shown below.

Table 26. CoVE Guest Get Device Link

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE Guest Get Device Connection Transcript (FID #1025)

```
struct sbiret sbi_covg_get_device_connection_transcript(unsigned long
device_if_id,
                                                    unsigned long
transcript_element_id,
                                                    unsigned long
transcript_element_parameter,
                                                    unsigned long
data_addr_out,
                                                    unsigned long
data_size);
```

Get a specific element from the device connection transcript, or the whole transcript.

The `transcript_element_id` specifies which part of the device connection transcript the TVM is requesting. It must be one of the following values:

```
enum sbi_covg_transcript_element {
    /* Full device connection transcript */
    FULL_TRANSCRIPT = 0,

    /*
     * Reconstructed certificate chain for the physical device hosting
     * `device_if_id`, from the transcript `CERTIFICATE` SPDM responses.
     * The TSM returns the certificate chain in the form of the <<SPDM>>
     * defined `certificate chain format`.
     * The `transcript_element_parameter` is set to a certificate chain slot
     * ID and thus must be between 0 and 7 inclusive.
     */
    DEVICE_CERTIFICATE = 1,

    /*
     * SPDM measurement messages for the physical device hosting
     * `device_if_id`.
     * This is the concatenation of all `{GET_MEASUREMENTS, MEASUREMENTS}`
     * pairs that are exchanged between the SPDM measurement requester and the
     * responder. Only the last `MEASUREMENTS` shall include the digital
```

```

    * signature of the measurement transcript.
    */
    DEVICE_MEASUREMENT = 2,

    /*
    * SPDM `VCA` (Version-Capabilities-Algorithms) for the physical device
    * hosting `device_if_id`.
    * This is the concatenation of the `GET_VERSION`, `VERSION`,
    * `GET_CAPABILITIES`, `CAPABILITIES`, `NEGOTIATE_ALGORITHMS` and
    * `ALGORITHMS` SPDM messages from the transcript.
    */
    VCA = 3,

    /*
    * SPDM session messages for the physical device hosting
    * `device_if_id`.
    * This is the concatenation of the `KEY_EXCHANGE`, `KEY_EXCHANGE_RSP`,
    * `FINISH`, `FINISH_RSP` SPDM messages from the transcript.
    * It includes the `TerminationPolicy`, and `HeartbeatPeriod` etc.
    */
    SESSION = 4,

    /*
    * TDISP messages for the physical device hosting
    * `device_if_id`.
    * This is the concatenation of the `TDISP_GET_VERSION`, `TDISP_VERSION`,
    * `TDISP_GET_CAPABILITIES` and `TDISP_CAPABILITIES` TDISP messages from
    * the transcript.
    * It includes the `TSM_CAPS`, `DSM_CAPS`, `REQ_MSGS_SUPPORTED`, and
    * `LOCK_INTERFACE_FLAGS_SUPPORTED`, etc.
    */
    TDISP_VC = 5,
}

```

`data_addr_out` must be 4KB-aligned.

The possible error codes returned in `sbiret.error` are shown below.

Table 27. CoVE Guest Get Device Connection Transcript

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_INVALID_PARAMS	Invalid parameter value provided for <code>device_if_id</code> or <code>transcript_element_id</code> .
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE Guest Get Device Measurements (FID #1026)

```

struct sbiret sbi_covg_get_device_measurements(unsigned long device_if_id,

```



```

nonce_addr,                                     unsigned long
msmt_req_attr,                                 unsigned long
msmt_addr_out,                                unsigned long
msmt_size);                                   unsigned long

```

Gets the measurements of the physical device hosting **device_if_id**.

The TVM can call this interface multiple times to get the latest device measurement.

The TSM returns the measurements in the form of the complete, signed [SPDM] measurement transcript, including VCA and all {GET_MEASUREMENTS, MEASUREMENTS} pairs that are exchanged between the SPDM measurement requester and the responder. Only the last MEASUREMENTS shall include the digital signature of the measurement transcript.

The **nonce_addr** parameter points at an optional 32 bytes long buffer holding a cryptographic nonce.

For any non zero value, the nonce is used as the SPDM GET_MEASUREMENTS request Nonce field. When set to 0x0, the TSM ignores this argument and generates a nonce on behalf of the TVM.

msmt_req_attr is used as the measurement request attributes in SPDM GET_MEASUREMENT request param1 field. Only RawBitStreamRequested bit is valid and the rest bits are ignored. The last GET_MEASUREMENT request must set SignatureRequested bit to request the digital signature of the measurement transcript.

Both **msmt_addr_out** and **nonce_addr** must be 4KB-aligned.

The possible error codes returned in **sbiret.error** are shown below.

Table 28. CoVE Guest Get Device Measurements

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_FAILED	The operation failed for unknown reasons.

9.3.2. Device Interface Management

Function: CoVE Guest Get Interface Report (FID #1027)

```

struct sbiret sbi_covg_get_interface_report(unsigned long device_if_id
                                             unsigned long
if_report_out,
                                             unsigned long
if_report_size);

```

Gets the TDISP interface report for the device interface.

The TSM returns the interface report, as defined by the [TDISP] TDI Report Structure, at the

`if_report_out` address.

`if_report_out` must be 4KB-aligned.

The possible error codes returned in `sbiret.error` are shown below.

Table 29. CoVE Guest Get Interface Report

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE Guest Get Interface State (FID #1028)

```
struct sbiret sbi_covg_get_interface_state(unsigned long device_if_id);
```

Gets the TDISP state for the device interface.

The TVM calls this function to verify that a bound device interface is in the TDISP `RUN` state.

Returns the interface state through `sbiret.value`.

```
enum sbi_covg_interface_state {
    /* TDISP CONFIG_UNLOCKED */
    Unlocked,

    /* TDISP CONFIG_LOCKED */
    Locked,

    /* TDISP RUN */
    Running,

    /* TDISP ERROR */
    Error,
};
```

The possible error codes returned in `sbiret.error` are shown below.

Table 30. CoVE Guest Get Interface State

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE Guest Map Interface MMIO (FID #1029)

```
struct sbiret sbi_covg_map_interface_mmio(unsigned long device_if_id
                                           unsigned long gpa_addr,
```

```
unsigned long hpa_addr,
unsigned long size);
```

Maps a TVM MMIO region (from `gpa_address`, `size` bytes long) to a **TDISP**-reported physical region (`hpa_addr`).

The TVM uses that function to verify from the TSM that all the device interface MMIO regions exposed by the host Supervisor Domain manager are correctly mapped to the trusted **TDISP**-reported MMIO regions. The TSM will enable those mappings if the TVM calls the starts the device interface through the `sbi_covg_start_interface` function.

All of `gpa_addr`, `hpa_addr` and `size` values must be 4KB-aligned.

The possible error codes returned in `sbiret.error` are shown below.

Table 31. CoVE Guest Map Interface MMIO

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE Guest Start Interface (FID #1030)

```
struct sbiret sbi_covg_start_interface(unsigned long device_if_id);
```

Starts a bound device interface.

The TVM calls this function in order to accept a bound device interface into its TCB. While servicing this request, the TSM moves the device interface **TDISP** state from **CONFIG_LOCKED** to **RUN**.

After this calls completes successfully, the device interface I/O is ready and available for the bound TVM.

The possible error codes returned in `sbiret.error` are shown below.

Table 32. CoVE Guest Start Interface

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_ALREADY_STARTED	The device interface is already started.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE Guest Stop Interface (FID #1031)

```
struct sbiret sbi_covg_stop_interface(unsigned long device_if_id);
```

Stops a bound device interface.

The TVM calls this function for either removing a bound device interface from its TCB or initially rejecting it.

After this calls completes successfully, the device interface and the TVM are no longer bound together.

The possible error codes returned in `sbiret.error` are shown below.

Table 33. CoVE Guest Stop Interface

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_ALREADY_STOPPED	The device interface is already stopped.
SBI_ERR_FAILED	The operation failed for unknown reasons.

9.4. CoVE IO TSM Extension (EID #0x434F5654 "COVT")

9.4.1. PCIe Root Port IDE Management

Function: CoVE TSM Program PCIe Root Port IDE Key (FID #1024)

```
struct sbiret sbi_covt_program_rp_ide_key(unsigned long rp_id,
                                         unsigned long key_desc,
                                         unsigned long key_data_addr);
```

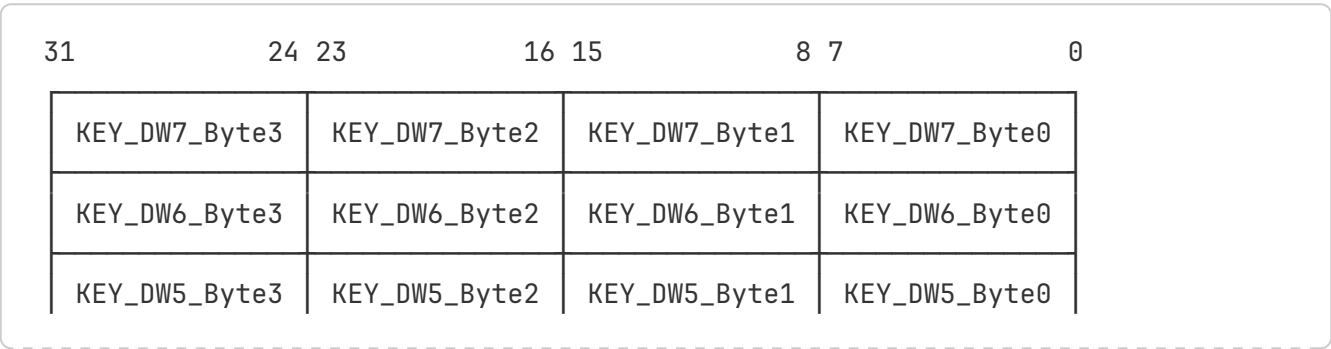
Programs a PCIe IDE key into the `rp_id` PCIe Root Port. This is functionally equivalent to the PCIe IDE KM `KEY_PROG` command.

The `key_desc` IDE key descriptor must be structured as follows:

Table 34. CoVE IDE Key Descriptor

Byte Offset	Field	Size (bytes)	Description
00h	Stream_ID	1	IDE selective stream ID
01h	Sub_Stream_ID	1	IDE sub-stream ID
02h	Key_Set	1	IDE Key set. This must be 0 or 1, all other values are invalid.
03h	Tx_Rx	1	Transmitter key (0) or receiver key (1). All other values are invalid.

The IDE key and IV data at `key_data_addr` must follow the same byte order and structure as the `KEY_PROG` data object payload, as described in the **IDE Key Management (IDE_KM)** section of the PCIe specification:



KEY_DW4_Byte3	KEY_DW4_Byte2	KEY_DW4_Byte1	KEY_DW4_Byte0
KEY_DW3_Byte3	KEY_DW3_Byte2	KEY_DW3_Byte1	KEY_DW3_Byte0
KEY_DW2_Byte3	KEY_DW2_Byte2	KEY_DW2_Byte1	KEY_DW2_Byte0
KEY_DW1_Byte3	KEY_DW1_Byte2	KEY_DW1_Byte1	KEY_DW1_Byte0
KEY_DW0_Byte3	KEY_DW0_Byte2	KEY_DW0_Byte1	KEY_DW0_Byte0
IFV_DW1_Byte3	IFV_DW1_Byte2	IFV_DW1_Byte1	IFV_DW1_Byte0
IFV_DW0_Byte3	IFV_DW0_Byte2	IFV_DW0_Byte1	IFV_DW0_Byte0

The initial state of a successfully programmed key is the **Programmed** state.

Calling `sbi_covt_program_rp_ide_key()` on a key that is in the **Programmed** state overwrites the existing key content.

Overwriting a key that is in the **Enabled** state (i.e. enabled through the `sbi_covt_set_go_rp_ide_key()` function) returns an **SBI_INVALID_STATE** error.

Table 35. CoVE TSM Program PCIe Root Port IDE Key

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_INVALID_ADDRESS	<code>key_data_addr</code> is invalid.
SBI_ERR_INVALID_PARAM	<code>key_desc</code> or <code>rp_id</code> is invalid. The key descriptor can be invalid if the stream ID is unknown or the sub-stream ID is invalid.
SBI_ERR_INVALID_STATE	The key is in the Enabled state and can not be programmed.
SBI_ERR_IO	The key can not be transferred or stored.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE TSM Clear PCIe Root Port IDE Key (FID #1025)

```
struct sbiret sbi_covt_clear_rp_ide_key(unsigned long rp_id,
                                       unsigned long stream_id);
```

Clears all previously programmed IDE key for `stream_id` in the `rp_id` PCIe Root Port.

If any `stream_id` bound IDE key is in the **Enabled** state (i.e. enabled through the `sbi_covt_set_go_rp_ide_key()` function), this function returns an **SBI_INVALID_STATE** error.

Once cleared from a Root Port, a key is no longer tracked by the RDSM and its status is **Unknown**.

Table 36. CoVE TSM Clear PCIe Root Port IDE Key

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_INVALID_PARAM	rp_id or stream_id is invalid.
SBI_ERR_INVALID_STATE	At least one stream_id key is not in the Programmed state.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE TSM Enable PCIe Root Port IDE Key (FID #1026)

```
struct sbiret sbi_covt_set_go_rp_ide_key(unsigned long rp_id,
                                         unsigned long key_desc);
```

Enables a previously programmed IDE key for the **rp_id** PCIe Root Port. This is functionally equivalent to the PCIe IDE KM **K_SET_G0** command.

Once enabled, a key status moves to the **Enabled** state.

If the enabled key is a transmitter key, the **rp_id** Root Port will use the key content to encrypt transmitted PCIe TLPs for the corresponding stream and sub-streams combination.

If the enabled key is a receiver key, the **rp_id** Root Port prepares for receiving and decrypting PCIe TLPs on the the corresponding stream and sub-streams combination.

Once enabled, a key can not be cleared or reprogrammed. It must be disabled first.

Table 37. CoVE TSM Enable PCIe Root Port IDE Key

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_INVALID_PARAM	key_desc or rp_id is invalid.
SBI_ERR_INVALID_STATE	The key is not in the Programmed state and can not be enabled.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE TSM Disable PCIe Root Port IDE Tx Key (FID #1027)

```
struct sbiret sbi_covt_set_stop_rp_ide_key(unsigned long rp_id,
                                             unsigned long key_desc);
```

Disables a previously enabled IDE key for the **rp_id** PCIe Root Port. This is functionally equivalent to the PCIe IDE KM **K_SET_STOP** command.

Once disabled, a key status moves back to the **Programmed** state.

Table 38. CoVE TSM Disable PCIe Root Port IDE Key

Error code	Description
SBI_SUCCESS	The operation completed successfully.

Error code	Description
SBI_ERR_INVALID_PARAM	key_desc or rp_id is invalid.
SBI_ERR_INVALID_STATE	The key is not in the Enabled state and can not be disabled.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE TSM PCIe Root Port IDE Key Status (FID #1028)

```
struct sbiret sbi_covt_rp_ide_key_status(unsigned long rp_id,
                                         unsigned long key_desc);
```

Gets the status of a PCIe Root Port IDE key.

Returns the key status through **sbiret.value**.

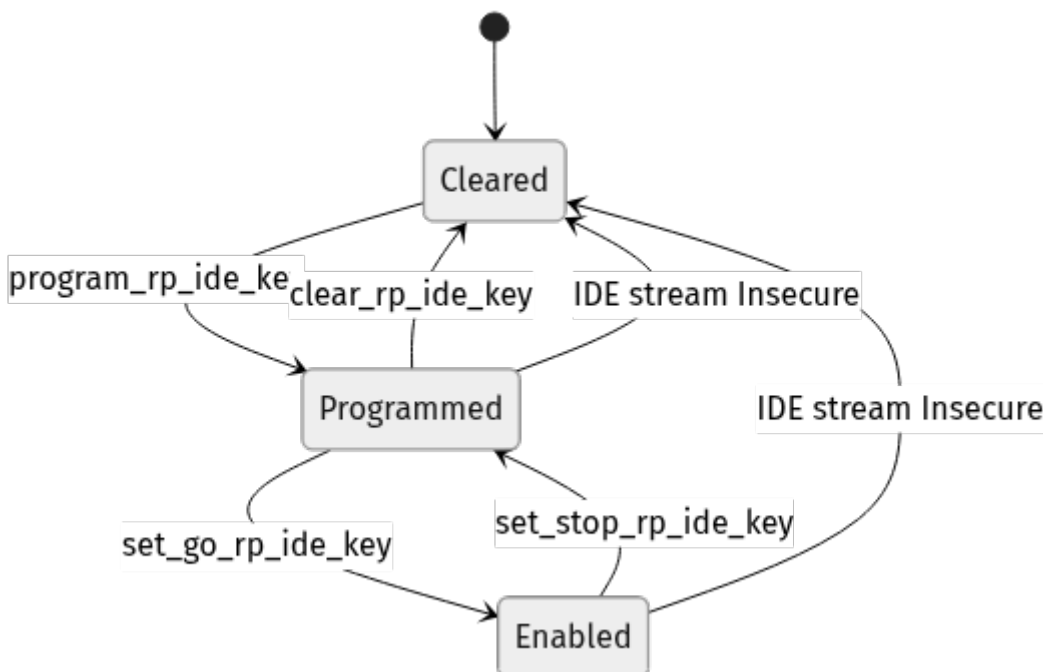
A key status must be one of the following values:

```
enum sbi_covt_ide_key_status {
    /* The key is not programmed in the Root Port */
    Cleared,

    /* The key is programmed but it is not enabled */
    Programmed,

    /* The key is programmed and enabled */
    Enabled,
};
```

When a selective IDE Stream State Machine transitions to the **Insecure** state from the **Secure** one, all associated keys for that stream move to the **Cleared** state:



The possible error codes returned in **sbiret.error** are shown below.

Table 39. CoVE TSM PCIe Root Port IDE Key Status

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_INVALID_PARAM	key_desc or rp_id is invalid.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Function: CoVE TSM Program PCIe Root Port IDE Stream Extended Capability (FID #1029)

```
struct sbiret sbi_covt_program_rp_ide_stream(unsigned long rp_id,
                                             unsigned long ide_stream_desc);
```

Programs the IDE Extended Capability of PCIe Root Port **rp_id** for the IDE stream described in **ide_stream_desc**.

Table 40. CoVE TSM Program PCIe Root Port IDE Stream

Error code	Description
SBI_SUCCESS	The operation completed successfully.
SBI_ERR_INVALID_PARAM	ide_stream_desc is invalid. The IDE stream descriptor can be invalid if the stream ID is unknown or if the descriptor data is inconsistent.
SBI_ERR_FAILED	The operation failed for unknown reasons.

Chapter 10. Summary Listing of CoVE-IO Functions

10.1. Summary of CoVE IO Host Extension (COVH)

sbi_covh_connect_device	The TSM connects to a physical device by establishing a secure SPDm session with the DSM. The TSM also configures the PCIe IDE link between that device and its upstream Root Port. The TSM must configure the IDE link according to the provided IDE stream description.
sbi_covh_disconnect_device	Disconnects the TSM from a physical device. The TSM stops all TVM-bound interfaces, clears the IDE link and keys, and terminates the secure SPDm session with the DSM.
sbi_covh_bind_interface	Binds a TVM and a TDI together. Upon success, the device interface moves to the TDISP CONFIG_LOCKED state.
sbi_covh_unbind_interface	Unbinds a TDI from a TVM. Upon success, the device interface moves back to the TDISP CONFIG_UNLOCKED state.
sbi_covh_add_tvm_interface_region	Adds a TDI MMIO region into a TVM address space. The TSM creates uncachable G-stage mappings for the region, which are enabled only after the TVM accepts the TDI (through the sbi_covg_start_interface SBI).
sbi_covh_reclaim_tvm_interface_region	Reclaims a TDI MMIO region previously added to a TVM address space. If the interface is still bound, the TSM unbinds it first.
sbi_covh_spdm_key_update	Performs an SPDm key update for a SPDm connection between the TSM and a connected device.
sbi_covh_ide_key_refresh	Performs an IDE key refresh for a connected device IDE stream.
sbi_covh_spdm_heartbeat	Performs an SPDm session heartbeat for a SPDm connection between the TSM and a connected device.
sbi_covh_get_device_connection_transcript	Passes the device connection transcript data back to the TSM for a given connected device, allowing the TSM to reconstruct the connection state. The TSM must verify the transcript's digest against its internal transcript digests map to ensure integrity.

10.2. Summary of CoVE IO Guest Extension (COVG)

sbi_covg_get_device_link	Gets the status of the link between a physical device hosting a TDI bound to a TVM. This status covers both the SPDm and IDE links.
sbi_covg_get_device_connection_transcript	Retrieves a specific element from the device connection transcript (such as certificates, measurements, or session keys) or the full transcript.
sbi_covg_get_device_measurements	Gets the measurements of a physical device hosting a TDI bound to a TVM. The TSM returns the complete, signed SPDm measurement transcript.
sbi_covg_get_interface_report	Gets the TDISP interface report for a TDI as defined by the TDISP TDI Report Structure (See [PCIe]).
sbi_covg_get_interface_state	Gets the TDISP state (Unlocked , Locked , Running , or Error) for a TDI.
sbi_covg_map_interface_mmio	Maps a TVM MMIO region to a TDISP -reported physical region. This allows the TVM to verify that the MMIO regions exposed by the host are correctly mapped to trusted hardware regions.
sbi_covg_start_interface	Starts a bound TDI, moving its TDISP state from CONFIG_LOCKED to RUN . After this, the interface is ready for the TVM.
sbi_covg_stop_interface	Stops a bound TDI. The TVM calls this to remove an interface from its TCB or to reject it.

10.3. Summary of CoVE IO TSM Extension (COVT)

sbi_covt_program_rp_ide_key	Programs a PCIe IDE key into a PCIe Root Port. Equivalent to the PCIe IDE KM KEY_PROG command.
sbi_covt_clear_rp_ide_key	Clears all previously programmed IDE keys for a specific IDE stream managed by a PCIe Root Port.
sbi_covt_set_go_rp_ide_key	Enables a previously programmed IDE key for a PCIe Root Port. Equivalent to the PCIe IDE KM K_SET_60 command.
sbi_covt_set_stop_rp_ide_key	Disables a previously enabled IDE key for a PCIe Root Port. Equivalent to the PCIe IDE KM K_SET_STOP command.
sbi_covt_rp_ide_key_status	Gets the status of a PCIe Root Port IDE key (Cleared, Programmed, or Enabled).
sbi_covt_program_rp_ide_stream	Programs the IDE Extended Capability of a PCIe Root Port according to the provided IDE stream description.

References

- [CoVE] RISC-V Confidential VM Extension, github.com/riscv-non-isa/riscv-ap-tee
- [CC] Confidential Computing, confidentialcomputing.io/about/
- [PCI-SIG] PCI SIG, pcisig.com/
- [DMTF] DMTF, www.dmtf.org/
- [SPDM] Security Protocol and Data Model Specification, www.dmtf.org/dsp/DSP0274
- [SecuredSPDM] Secured Messages using SPDM Specification, www.dmtf.org/dsp/DSP0277
- [PCIE] PCI Express Base Specification, members.pcisig.com/wg/PCI-SIG/document/18363
- [TDISP] TEE Device Interface Protocol ECN, members.pcisig.com/wg/PCI-SIG/document/18268?uploaded=1
- [AEADLimit] IDE_KM GETKEY and AEAD Limit ECN, members.pcisig.com/wg/PCI-SIG/document/22652
- [CMA_PQC] CMA PQC support ECN, members.pcisig.com/wg/PCI-SIG/document/23024
- [IOMMU] RISC-V IOMMU, github.com/riscv-non-isa/riscv-iommu
- [RATS] IETF RFC 9334 Remote ATtestation procedureS (RATS) Architecture, datatracker.ietf.org/doc/rfc9334/
- [Smmtt] RISC-V Supervisor Domain Access Protection, github.com/riscv/riscv-smmtt
- [Smmpt] RISC-V Supervisor Mode Memory Protection Table, github.com/riscv/riscv-smmpt/blob/main/chapter3.adoc
- [IO-MPT] RISC-V Supervisor Mode I/O Memory Protection Table, github.com/riscv/riscv-smmpt/blob/main/chapter6.adoc